浙江大学
ZHEJIANG UNIVERSITY

组 合 优 化

浙江大学 谈之奕



浙江大学
ZHEJIANG UNIVERSITY

计算复杂性初步



算法与计算复杂性

- 算法（algorithm）
  - 在有限步骤内求解某一问题的一组含义明确的可以完全机械执行的规则
  - 一系列将输入转换为输出的计算步骤
- 算法的品质
  - 衡量算法品质的主要因素是其执行时间的长短和所需存储容量的多少
- 计算复杂性（computational complexity）
  - 确定一个问题的所有可能的算法解的最优效率或最优效率的界限
  - 理论计算机科学的一个重要分支，主要研究如何确定一个问题的所有可能的算法解的最优效率或最优效率的界限



Muḥammad ibn Mūsā al-Khwārizmī
(中译名：花拉子米)
(拉丁译名：**Algorithmi**)
(约780-约850)
波斯数学家

问题与实例

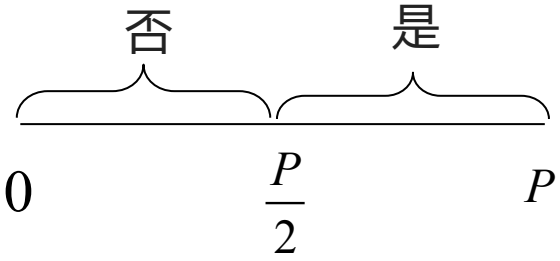
- 问题与实例
  - 问题（**problem**）指需要回答的一般性提问，通常带有若干参数；对一问题的所有参数指定具体值可得到该问题的一个实例（**instance**）
  - 判定问题（**decision problem**）：任一实例只有“是”、“否”两个可能答案的问题
- 背包问题及其判定形式
  - 背包问题：对由若干物品组成的集合，求它的一个子集，使得子集中物品大小之和不超过背包容量，价值之和最大
  - 背包问题的判定形式：对由若干物品组成的集合，是否存在物品集的一个子集，使得子集中物品大小之和不超过背包容量，价值之和不小于某个阈值
  - 实例：n 个物品，物品 j 的价值为 p_j，大小为 w_j，$j=1,2,\cdots,n$，背包容量为 C。对判定问题，实例描述中还需增加阈值 M

优化问题的判定形式



组合优化 MATH T

- 优化问题与其判定形式在计算复杂性上的等价性
  - 求解优化问题的算法可直接求解其判定形式
  - 通过多次求解判定形式，可以求得优化问题的最优解或最优值
- 背包问题及其判定形式
  - 设存在一背包问题的算法，可求得实例最优值为 M^*
    - 若 $M^* < M$，则判定形式实例答案为“否”
    - 若 $M^* \geq M$，则判定形式实例答案为“是”
  - 设存在一背包问题判定形式的算法 A，可求得任意判定形式实例的解
    - 背包问题实例的最优值 $M^* \in [0, P]$，其中 $P = \sum_{j=1}^{n} p_j$
    - 对阈值为 $\frac{P}{2}$ 的判定形式实例，调用算法 A 求解
      - 若输出“是”，则最优值 $M^* \in \left[\frac{P}{2}, P\right]$
      - 若输出“否”，则最优值 $M^* \in \left[0, \frac{P}{2}\right)$
      - 经 $O(\log P)$ 次调用算法 A，可求得背包问题实例的最优值

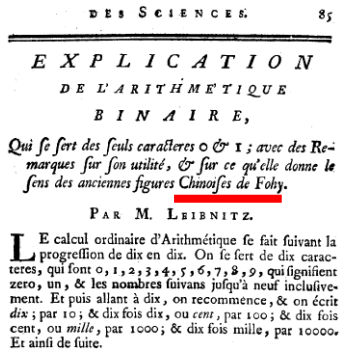


规模

- 实例的规模
  - 规模（size）：描述一实例所需的计算机存储单元数
  - 在计算机中，常用二进制表示整数，存储整数 n 所需字节数为 $\lfloor \log_2 n \rfloor + 1$
- 背包问题实例规模
  - 背包问题实例：n 个物品，物品 j 的价值为 p_j，大小为 w_j，$j = 1, 2, \cdots, n$，背包容量为 C
  - 背包问题实例规模

$$\sum_{j=1}^{n}\left(\lfloor \log_2 p_j \rfloor + 2\right) + \sum_{j=1}^{n}\left(\lfloor \log_2 w_j \rfloor + 2\right) + \left(\lfloor \log_2 C \rfloor + 2\right)$$

《周易》，中国儒家典籍，六经之一，成书于周。全书由64卦组成。每卦由6个“爻”排列而成。每个爻为阳爻“——”或阴爻“— —”两种符号之一。《周易》虽属占书，但在其神秘的形式中蕴含着较深刻的理论思维和辩证观念



DES SCIENCES. 85

EXPLICATION
DE L'ARITHMETIQUE
BINAIRE,

Qui ſe ſert des ſeuls caracteres 0 *&* 1 *; avec des Remarques ſur ſon utilité, & ſur ce qu'elle donne le ſens des anciennes figures Chinoiſes de Fohy.*

PAR M. LEIBNITZ.

LE calcul ordinaire d'Arithmétique ſe fait ſuivant la progreſſion de dix en dix. On ſe ſert de dix caracteres, qui ſont 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, qui ſignifient zero, un, & les nombres ſuivans juſqu'à neuf inclusivement. Et puis allant à dix, on recommence, & on écrit *dix* ; par 10 ; & dix fois dix, ou *cent*, par 100 ; & dix fois cent, ou *mille*, par 1000 ; & dix fois mille, par 10000. Et ainſi de ſuite.



Gottfried Wilhelm Leibniz (1646-1716)
德国哲学家、数学家

Leibniz G, Explanation of binary arithmetic, which uses only the characters 1 and 0, with some remarks on its usefulness, and on the light it throws on the ancient Chinese figures of Fu Xi, *Memoires de mathématique et de physique de l'Académie royale des sciences*, Académie Royale des Sciences, 1703



《周易》原书前6卦



北宋哲学家邵雍按自己的理解重排64卦中的前6卦

规模



组合优化

MATH T

- 离散函数的量阶
  - $f(n)=\mathrm{O}(g(n))$：存在常数$C>0$和$n_0>0$，使得对任意$n>n_0$，$|f(n)|\le C|g(n)|$
  - $f(n)=\Omega(g(n))$：存在常数$C>0$和$n_0>0$，使得对任意$n>n_0$，$|f(n)|\ge C|g(n)|$
  - $f(n)=\Theta(g(n))$：$f(n)=\mathrm{O}(g(n))$ 且 $f(n)=\Omega(g(n))$
  - $f(n)=o(g(n))$：对任意$\varepsilon>0$，存在$n_0>0$，使得对任意$n>n_0$，$|f(n)|\le \varepsilon|g(n)|$
- 多项式相关函数
  - 函数 $f(n)$ 和 $g(n)$ 多项式相关，若 $f(n)=O(\mathrm{poly}(g(n)))$且$g(n)=O(\mathrm{poly}(f(n)))$
- 规模表达式的等价性

抓主要矛盾

  - 为消除编码方式和存储方式不同可能带来的影响，在多数与计算复杂性相关的讨论中，多项式相关的规模表达式均为合理的

Knuth DE. Big omicron and big omega and big theta. *ACM SIGACT News*, 8, 18-24, 1976.

规模



组合优化

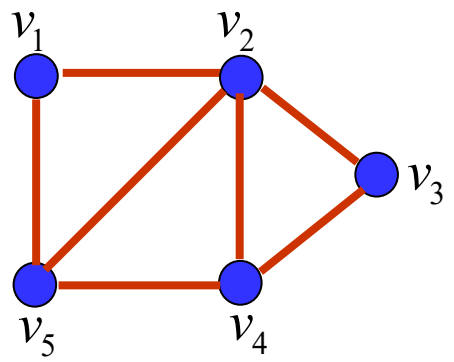
MATH T

- 背包问题
  - 背包问题实例 I 的规模可记为 $s_1(I)=\sum_{j=1}^{n}\left(\lfloor\log_2 p_j\rfloor+2\right)+\sum_{j=1}^{n}\left(\lfloor\log_2 w_j\rfloor+2\right)+\left(\lfloor\log_2 C\rfloor+2\right)$，也可记为 $s_2(I)=n+\log_2 B$，其中 $B=\max\left\{\max_{j=1,\cdots,n} p_j, \max_{j=1,\cdots,n} w_j, C\right\}$
    - $s_1(I)\le(2n+1)(\log_2 B+2)\le(n+\log_2 B)^2=s_2(I)^2$，$s_2(I)\le s_1(I)$
  - 背包问题实例的规模不能记为 $(2n+1)B$ 或 $O(n)$
- TSP问题
  - TSP问题实例 I 的规模可记为 $s_1(I)=\sum_{i,j}\left(\lfloor\log_2 c_{ij}\rfloor+2\right)=2n^2+\sum_{i,j}\lfloor\log_2 c_{ij}\rfloor$，也可记为 $s_2(I)=n+\lceil\log_2 L\rceil$，其中 $L=\max_{i,j} c_{ij}$
    - $s_1(I)=2n^2+\sum_{i,j}\lfloor\log_2 c_{ij}\rfloor\le 2n^2+n^2\lceil\log_2 L\rceil\le\left(n+\lceil\log_2 L\rceil\right)^3=\left(s_2(I)\right)^3$，$s_2(I)=n+\lceil\log_2 L\rceil\le 2n^2+\sum_{i,j}\lfloor\log_2 c_{ij}\rfloor=s_1(I)$
  - TSP问题实例规模不能表示成 n^2 或 $n+L$

最大数

- 最大数
  - 最大数：实例中描述中出现的最大整数
- 规模与最大数
  - 最大数可以是规模的指数函数，也可以是规模的多项式函数
    - TSP问题实例的最大数 $m(I)=\max\{n,L\}$，是实例规模 $s(I)=n+\lceil\log_2 L\rceil$ 的指数函数
    - Euler图问题实例的最大数 $m(I)=n$，是实例规模 $s(I)=n^2$ 的指数函数
  - 规模与最大数函数对的多项式相关性
    - $s_1(I)=O(\text{poly}(s_2(I)))$，$s_2(I)=O(\text{poly}(s_1(I)))$
    - $m_1(I)=O(\text{poly}(m_2(I),s_2(I)))$，$m_2(I)=O(\text{poly}(m_1(I),s_1(I)))$

可能出现“很大的数”



$$\begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{pmatrix}$$

图的邻接矩阵
(adjacency matrix)
表示

时间复杂度



组合优化

MATH T

- 时间复杂度
  - 在计算复杂性中，用算法执行过程中所需的加法、比较、赋值等基本操作次数表示算法所用的时间
  - 时间复杂度（**time complexity**）是关于实例规模 n 的一个函数 $f(n)$，它表示用该算法求解所有规模为 n 的实例中所需基本操作次数最多的那个实例的基本操作次数
  - 若一算法的时间复杂度 $f(n)=O(\mathrm{poly}(n))$，则称它为多项式时间算法，不能这样限制时间复杂度函数的算法称为指数时间算法
  - 若某算法的时间复杂度是实例规模 n 和最大数 B 的二元多项式函数 $f(n)=O(\mathrm{poly}(n,B))$，但不是实例规模 n 的（一元）多项式函数，则称它为伪多项式时间算法（pseudo-polynomial time algorithm）

6 5 3 1 8 7 2 4

冒泡排序（Bubble Sort）对 n 个数按从小到大的顺序进行排序，需要执行的读取、交换和比较次数至多为 $7.5n^2+0.5n+1$，至少为 $8n+1$，其时间复杂度为 $O(n^2)$

基本运算次数，非实际运行时间

相对规模，非绝对次数

最坏情况，非平均或最好情况

$2^{O(\mathrm{poly}(\log n))}$ **拟多项式 (quasi-polynomial)** $O((\ln n)^{\ln n})$

时间复杂度



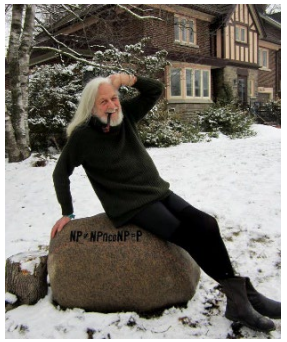
组合优化

MATH T

- 伪多项式时间算法
  - 若某类实例中不出现“很大的”数，伪多项式时间算法求解为多项式时间
    - 设 $f(I)=O(\text{poly}(s(I),m(I)))$
      - 若 $m(I)=O(p_1(s(I)))$，则 $f(I)=O(p_2(s(I)))$
      - 若 $m(I)=\Omega(2^{s(I)})$，则 $f(I)=\Omega(2^{s(I)})$
  - 满足 $m(I)=O(p_1(s(I)))$ 的“非数字问题”例举
    - 实例中不出现数：无权简单图上的问题
    - 实例中出现的数有大小限制：所有城市之间的距离为1或2等情形的TSP问题
- 背包问题的算法
  - 最大数 $L=\max\left\{\max_{j=1,\cdots,n} p_j, \max_{j=1,\cdots,n} w_j, C\right\}$，规模 $n+\lceil\log_2 L\rceil$
    - 时间复杂度为 $O(n^2)$，$O(n\log L)$ 等的算法都是多项式时间算法
    - 时间复杂度为 $O(n!)$，$O(2^n L)$，$O(n2^L)$ 等的算法都是指数时间算法
    - 时间复杂度为 $O(nL)$，$O(L\log n)$ 等的算法都是伪多项式时间算法
  - 若背包问题实例的规模记为 $n+L$，则时间复杂度为 $O(nL)$ 的算法将成为多项式时间算法

高效算法

- 高效算法
  - 结合函数增长速度和算法的实际运行效果，多项式时间算法可视为高效算法（**efficient algorithm**）或实用算法（**practical algorithm**），指数时间算法为不实用算法（**impractical algorithm**）
    - 多项式次调用多项式时间算法仍是多项式时间算法



Jack Edmonds (1934-) 加拿大Waterloo大学数学系教授



Kathie Cameron 加拿大Wilfrid Laurier大学理学院教授



Jeff Edmonds 加拿大York大学电气工程和计算机科学系教授

An explanation is due on the use of the words "efficient algorithm." ..... According to the dictionary, "efficient" means "adequate in operation or performance." .....There is an obvious finite algorithm, but that algorithm increases in difficulty exponentially with the size of the graph. It is by no means obvious whether or not there exists an algorithm whose difficulty increases only algebraically with the size of the graph.

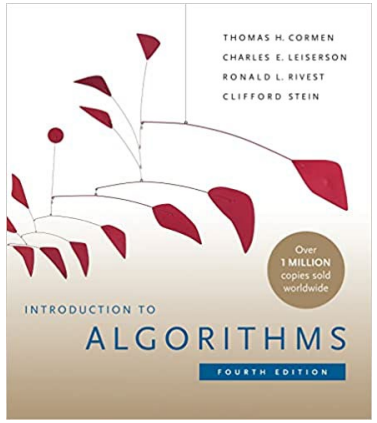
Edmonds J. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17, 449–467, 1965

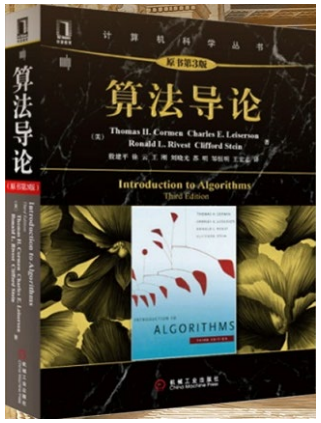
思考题

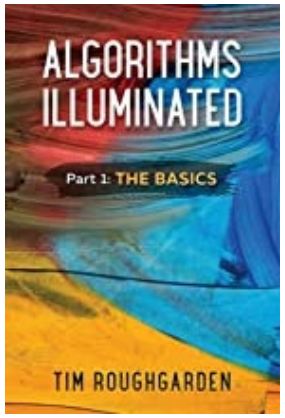
- 任务安排问题的规模
  - 现有 n 项任务，任务 j 所需时间为 p_j
    - 实例规模为 $n+\lceil \log_2 L \rceil$，其中 $L=\max\limits_{j=1,\cdots,n} p_j$
  - 现有 n 项任务，每项任务所需时间均为 p
    - 实例规模为 $\log n+\log p$
  - 时间复杂度为 $O(n\log n)$ 的问题对前者是多项式时间算法，对后者是指数时间算法

在讨论算法的时间复杂度时，实例规模表达式可在多项式相关范围内作适当简化，但不能通过“浪费”空间以“降低”算法的时间复杂度

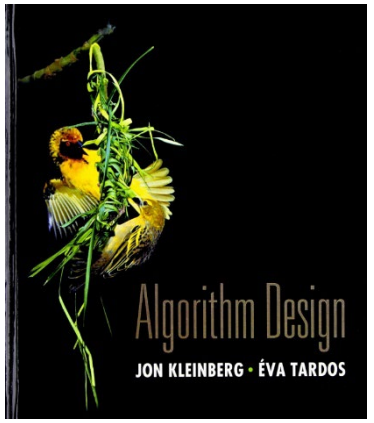
参考资料

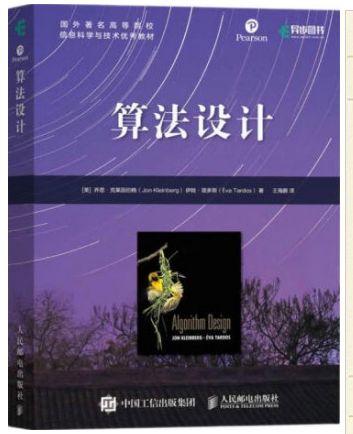


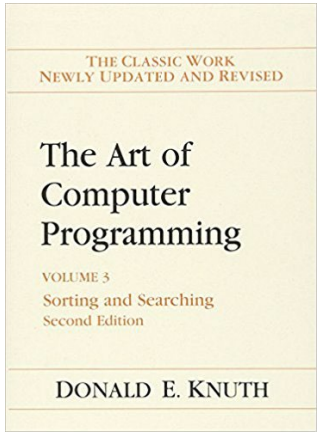












Cormen TH, Leiserson CE, Rivest RL, Stein C. *Introduction to Algorithms* (4th). MIT press, 2022（中译本：算法导论(第三版)，殷建平等译，机械工业出版社，2012.

Roughgarden T. *Algorithms Illuminated* (I-IV). Soundlikeyourself Publishing, 2017.（中译本：算法详解，徐波译，人民邮电出版社，2019.）

Kleinberg J, Tardos É. *Algorithm Design*. Pearson Education India, 2006（中译本：算法设计，张立昂、屈婉玲译，清华大学出版社，2007）

Knuth DE. *The Art of Computer Programming*. Addison-Wesley, 1968-2011.
Vol. 1. Fundamental Algorithms
Vol. 2. Seminumerical Algorithms
Vol. 3. Sorting and Searching
Vol. 4. Combinatorial Algorithms



Donald Ervin Knuth（高德纳）（1938- ）
美国计算机学家、1974年图灵奖得主

图灵机

- 图灵机
  - 图灵机是Turing提出的一种理想的计算机的数学模型，是可计算性和计算复杂性的基础
    - **Turing A, On computable numbers, with an application to the Entscheidungsproblem, *Proceedings of the London Mathematical Society*, S2-42, 230–265, 1936**
  - **Church–Turing Thesis**
    - 所有在某个合理的计算模型上可计算的函数都能用图灵机来计算



Alonzo Church
美国数学家、逻辑学家
(1903-1995)

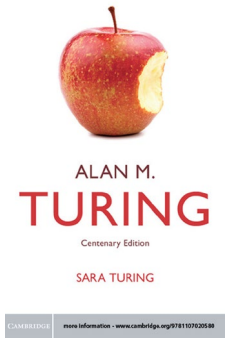


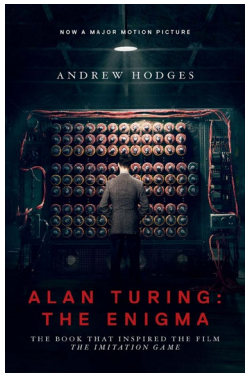
Alan Turing
英国数学家、计算机学家
(1912-1954)

美国计算机协会（**Association for Computing Machinery**）设有图灵奖（**A.M Turing Award**），奖励在计算机领域作出杰出贡献的学者，被誉为计算机界的诺贝尔奖。自1966至2024年，共有79位学者获得图灵奖









Turing S. *Alan M. Turing: Centenary Edition*, Cambridge University Press,2014

Hodges, A, *Alan Turing: The Enigma*, Princeton University Press, 2014

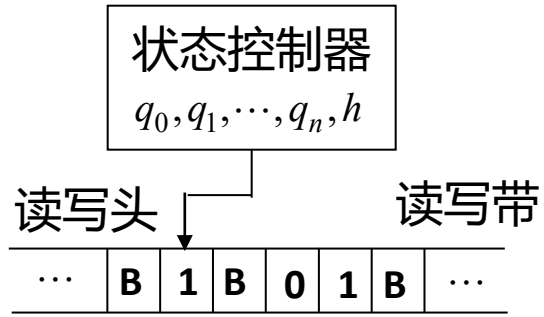
图灵机



组合优化

MATHT

- 图灵机的组成
  - 图灵机由一条或多条读写带、一个读写头、一个状态控制器和由有限条指令组成的程序构成
    - 读写带长度无限，分为多个方格，每个方格上可写入字母表（alphabet）中的一个符号
    - 读写头指向读写带上的一个方格，可左右移动，并进行读、写操作
    - 状态控制器包含有限个状态，任一时刻处于其中一个状态
    - 程序可用状态转移函数来表示
- 图灵机的瞬间像
  - 某时刻图灵机的当前状态、读写头所在位置、读写带上的非空白字符串构成瞬间像（configuration）



状态转移函数

方格上原符号　方格上新符号

$$\delta(q,\alpha)=(q',\beta,R/L)$$

当前状态　下一个状态　读写头移动规则

图灵机

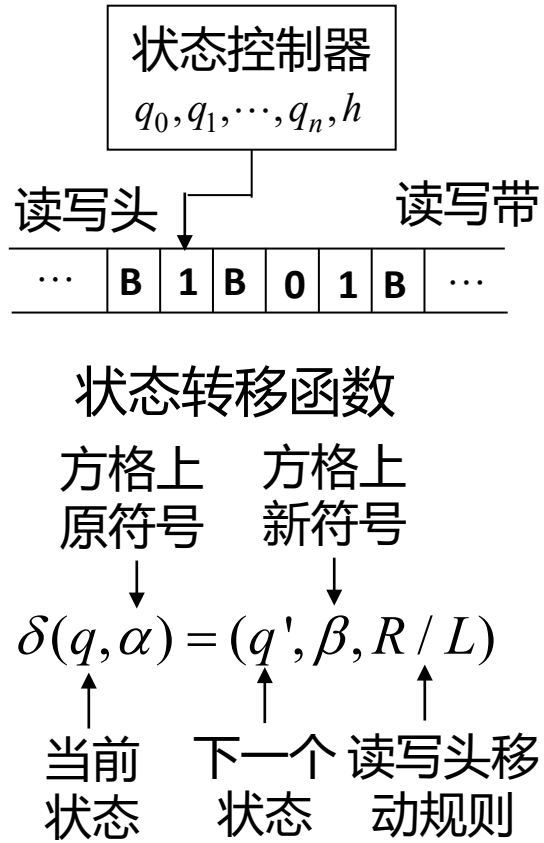


组合优化

MATH T

- 图灵机的运行
  - 图灵机的运行过程由一系列相继进行的步组成，每步由一个瞬间像变化到另一个瞬间像
    - 读取当前状态和读写头所指向的方格上的符号，获得以此为自变量的状态转移函数的函数值
    - 根据函数值，由当前状态转向下一个状态；读写头在所指向的方格上消去原符号，写上新符号；读写头向左或向右移动一个方格
- 图灵机的输入与输出
  - 图灵机运行开始时读写带上的字符串即为输入
  - 图灵机运行停止时读写带上的字符串即为输出
- 图灵机的计算时间与空间
  - 图灵机从运行开始到停止经过的步数为计算时间，从运行开始到停止读写头指向过的方格数为计算空间

对某些输入，图灵机可能永不停止或无法继续运行



投影



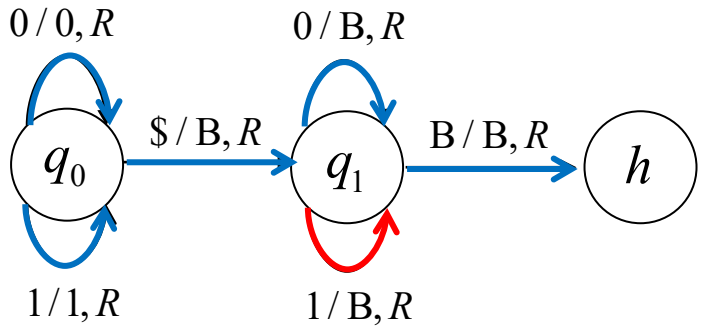
组合优化

MATHT

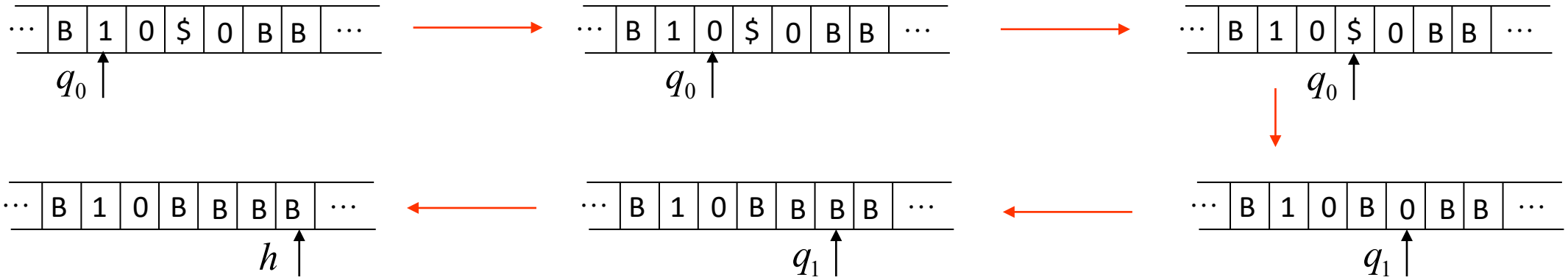
- 投影图灵机

δ	0	1	\$	B
q_0	$(q_0, 0, R)$	$(q_0, 1, R)$	(q_1, B, R)	
q_1	(q_1, B, R)	(q_1, B, R)		(h, B, R)

状态转移函数列表形式



状态转移图形式



DTM与NTM

- 确定性图灵机与非确定性图灵机
  - 确定性图灵机（**Deterministic Turing Machine，DTM**）的状态转移函数为单值函数，非确定性图灵机（**Nondeterministic Turing Machine，NTM**）状态转移函数可能为多值函数
  - 确定性图灵机是一种特殊的非确定性图灵机
  - 对给定的输入，确定性图灵机的运行过程是一条路，称为计算路。非确定性图灵机的运行过程可能是一棵树，称为计算树

非确定性图灵机不同于概率图灵机（Probabilistic Turing Machine，PTM），后者以一定概率从状态转移函数所有可能取值中选取一个

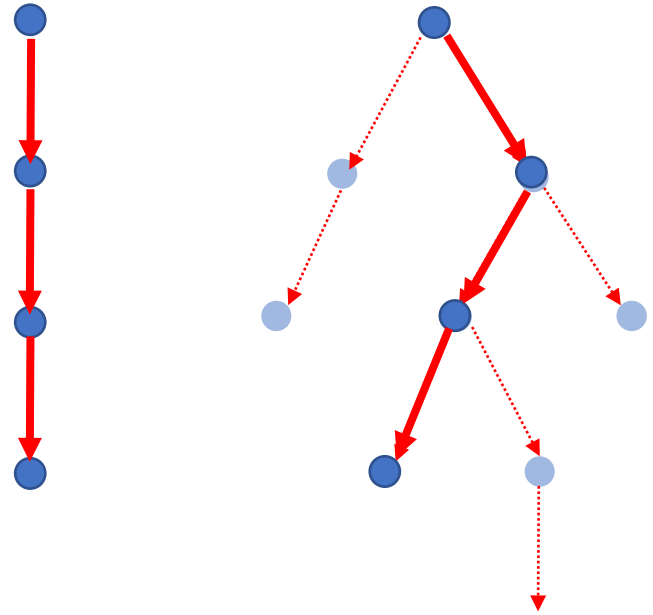
确定性图灵机

$$\delta(q,\alpha)=(q',\beta,R/L)$$

非确定性图灵机

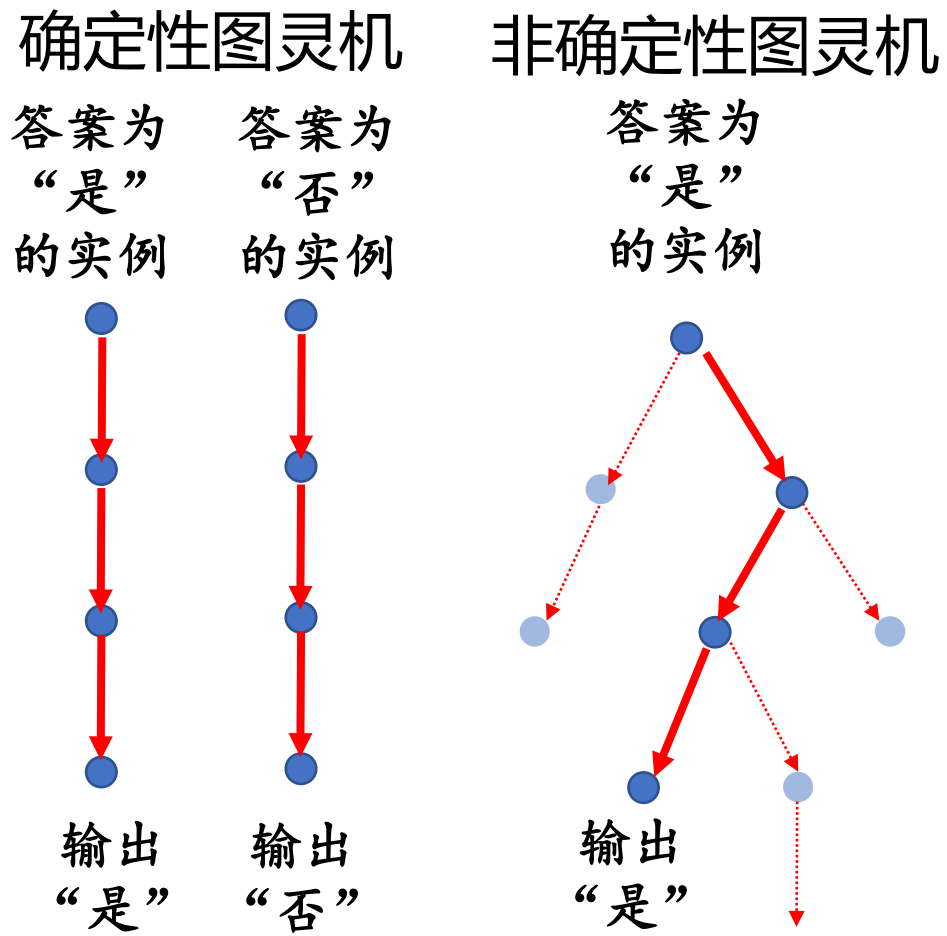
$$\delta(q,\alpha)=\{(q_1',\beta_1,R/L),(q_2',\beta_2,R/L)\}$$

确定性图灵机　非确定性图灵机



图灵机求解判定问题

- 确定性图灵机求解判定问题
  - 对每个答案为“是”的实例，图灵机运行停止并输出“是”，对每个答案为“否”的实例，图灵机运行停止并输出“否”
  - 计算时间为从运行开始到停止经过的步数
- 非确定性图灵机求解判定问题
  - 对每个答案为“是”的实例，图灵机的计算树中有一条路运行停止并输出“是”
  - 计算时间为这条路上从运行开始到停止经过的步数
- 图灵机与算法
  - 算法的基本操作均能通过图灵机经过多项式步运行实现



$\mathcal{P}$ 类与$\mathcal{NP}$类



组合优化

MATH T

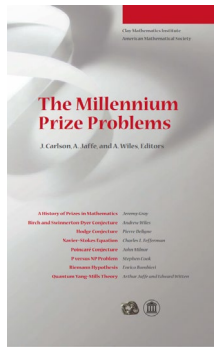
- $\mathcal{P}$ 类与$\mathcal{NP}$ 类
  - 确定性图灵机多项式时间可解问题类（polynomial solvable problem class），称为$\mathcal{P}$ 类，简记为$\mathcal{P}$
  - 非确定性图灵机多项式时间可解问题类（nondeterministic polynomial solvable problem class）称为$\mathcal{NP}$ 类，简记为$\mathcal{NP}$
- $\mathcal{P}$ vs. $\mathcal{NP}$ 问题
  - 由定义，$\mathcal{P} \subseteq \mathcal{NP}$
  - $\mathcal{P} = \mathcal{NP}$ 与 $\mathcal{P} \subset \mathcal{NP}$ 中何者成立

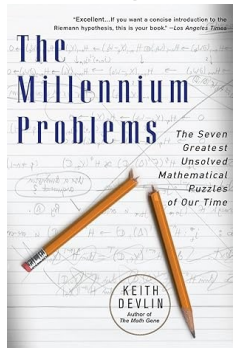
Millennium Problems

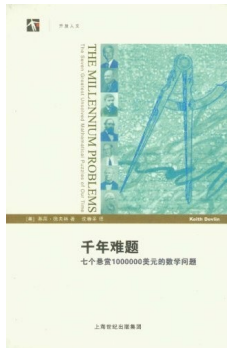
Yang–Mills and Mass Gap
Riemann Hypothesis
P vs NP Problem
Navier–Stokes Equation
Hodge Conjecture
Poincaré Conjecture
Birch and Swinnerton-Dyer Conjecture



Clay Mathematics Institute







Carlson J, Jaffe A, Wiles A (Eds.) *The Millennium Prize Problems*, AMS, 2006.

Devlin KJ, *The Millennium Problems: The Seven Greatest Unsolved Mathematical Puzzles of Our Time*, Basic Books , 2003.（中译本：沈崇圣译，上海科技教育出版社，2012）

$\mathcal{P}$ vs $\mathcal{NP}$ 问题



组合优化

MATH T

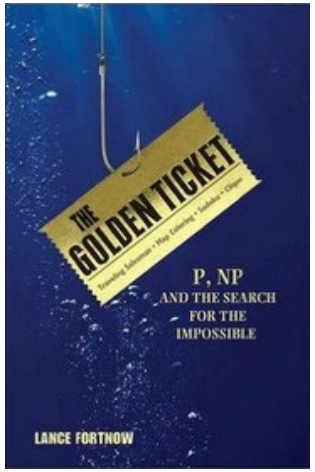
- $\mathcal{P}$ vs. $\mathcal{NP}$ 问题
  - 尽管 $\mathcal{P}$ vs. $\mathcal{NP}$ 问题尚未解决，但是目前普遍相信 $\mathcal{P} \neq \mathcal{NP}$ 成立，并在此假设下进一步研究$\mathcal{NP}$类内部的结构

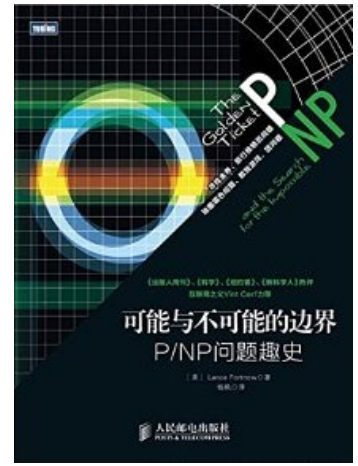
结论为何

	$\mathcal{P} \neq \mathcal{NP}$	$\mathcal{P} = \mathcal{NP}$	其他
2002	**61%**	**9%**	**30%**
2012	**83%**	**9%**	**8%**
2019	**88%**	**12%**	**0%**

何时解决

02-09	10-19	20-29	30-49	50-99	>2100	不知道
5%	12%	13%	15%	17%	17%	21%
	1%	11%	15%	26%	41%	6%
		22%	28%	16%	34%	0%





Fortnow L, *The Golden Ticket: P, NP, and the Search for the Impossible*, Princeton University Press, 2013. (中译本：可能与不可能的边界：P/NP问题趣史，杨帆译，人民邮电出版社，2014.)

Fortnow L. The status of the P versus NP problem. *Communications of the ACM*, 52(9): 78-86, 2009.

Gasarch WI . The Third P=? NP poll. *ACM SIGACT News*, 50(1), 38-59, 2019.

素数判定



组合优化

MATH T

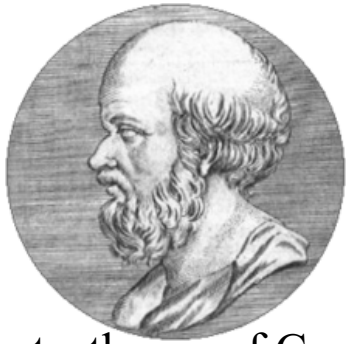
- 素数判定
  - 给定整数 n，判断 n 是否为素数
    - 实例规模为 $\log_2 n$，Eratosthenes筛法的时间复杂度为 $\Omega(\sqrt{n})$，是指数时间算法
    - AKS素数判定算法的时间复杂度为 $O(\log_2^{7.5} n \cdot \mathrm{poly}(\log\log n))$



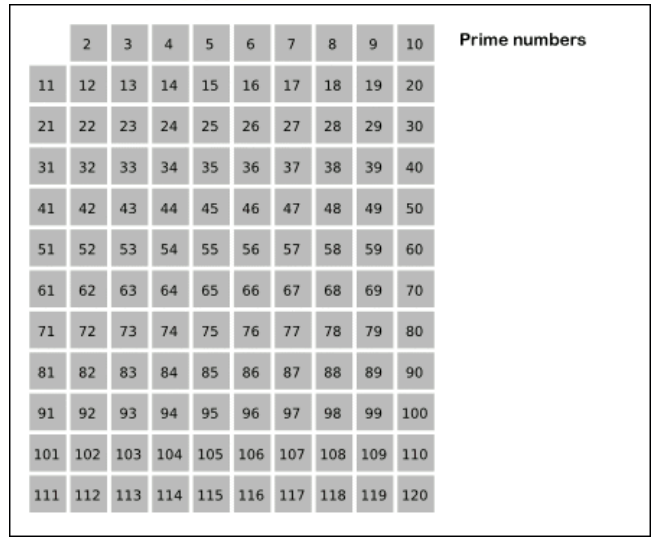


Manindra Agrawal时为印度理工学院坎普尔学院（Indian Institute of Technology Kanpur）计算机科学与工程系教授。Neeraj Kayal与Nitin Saxena在2002年时均为该系本科生，“Towards a Deterministic Polynomial-Time Primality Test”是他们参与的一项本科生科研项目

Agrawal M, Kayal N, Saxena N, PRIMES is in P. *Annals of Mathematics*, 160, 2, 781-793, 2004



Eratosthenes of Cyrene
(约公元前276-
约公元前194)
古希腊科学家



Here enter the heros of our story, off stage until now, the students Neeraj Kayal and Nitin Saxena. Both were members of the Indian team in the 1997 International Mathematical Olympiad. Studying computer science instead of mathematics because of better employment prospects, they found in complexity theory a way to continue working with mathematics on a high level.

Bornemann F, PRIMES Is in P: A Breakthrough for “Everyman”, Notices of the AMS, 2003.5, 545-552

NP类



组合优化

MATHT

- NP类的通俗定义
  - 非确定性算法多项式时间可求解（判定）问题类
    - 对任何一个答案为“是”的实例，算法能猜想出该实例的一个可行解
    - 该可行解的规模不超过实例规模的多项式函数
    - 能在实例规模的多项式时间内验证猜想是否正确
- 子集和问题（Subset Sum）
  - 给定正整数集 $A=\{a_1,a_2,\cdots,a_n\}$ 和数 B，问是否存在子集 $A_1\subseteq A$，使得 $\sum_{a_i\in A_1} a_i = B$
    - 确定性多项式时间算法：在多项式时间内求出满足要求的子集 A_1 可求解
    - 非确定性多项式时间算法 可验证
      - 猜出子集 $A_1=\{a_{i_1},a_{i_2},\cdots,a_{i_k}\}$，该集合可用长为 n 的0-1字符串表示
      - 通过 n 次加法和比较验证 A_1 中元素之和恰为 B

非确定性算法只是为研究而定义的一种理论算法模型，无法用目前的计算工具实现

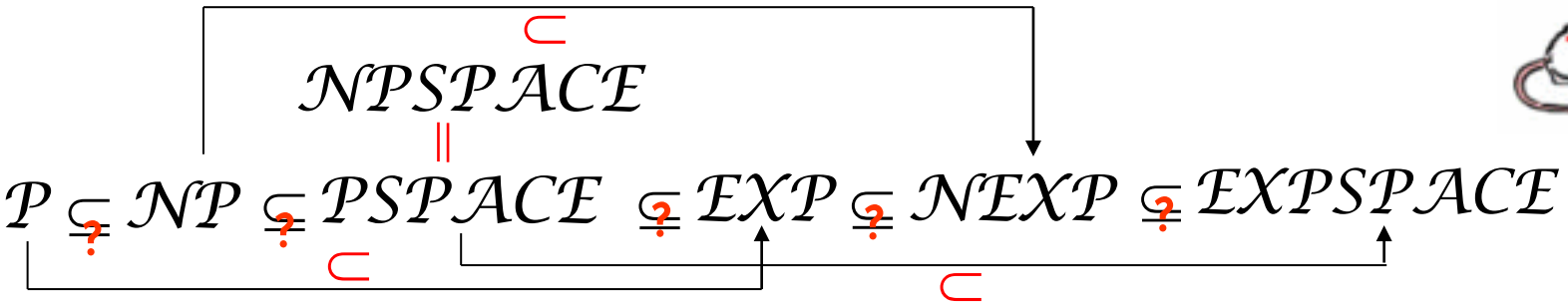
$\mathcal{NP}$类

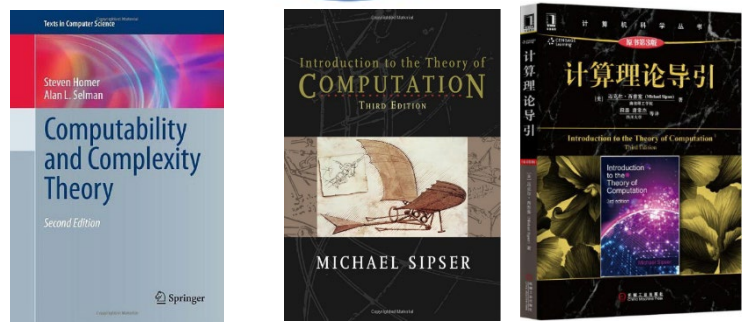


组合优化

MATH T

- 不在$\mathcal{NP}$ 类中的问题
  - 非判定问题
  - 不可判定问题（Undecidable Problem）
  - 非确定性算法多项式时间不可解问题
- 可能不在$\mathcal{NP}$ 类中的问题
  - 第 K 个最大子集和问题
    - 给定正整数集 $A=\{a_1,a_2,\cdots,a_n\}$ 和数 B，问是否存在 A 的 K 个子集 $A_1,A_2,\cdots,A_K$，使得 $\sum_{a_i\in A_i} a_i \le B, i=1,\cdots,K$
    - 实例规模 $n+\max\limits_{i=1,\cdots,n}\log a_i+\log B+\log K$，可行解的规模 Kn





Homer S, Selman AL. *Computability and Complexity Theory*, Springer, 2011.

Sipser M. *Introduction to the Theory of Computation* (3rd). Cengage Learning, 2012. (中译本：计算理论导引. 段磊、唐常杰译, 机械工业出版社，2015)



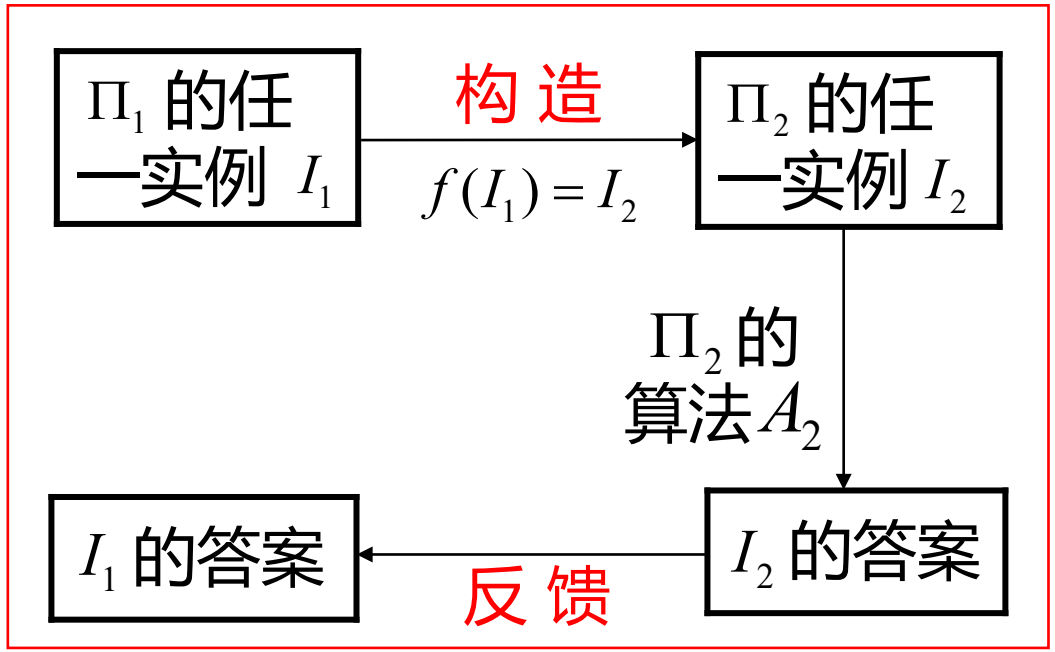
Complexity Zoo

complexityzoo.net/Complexity_Zoo

satoshihada.github.io/complexity-zoo/

$\mathcal{NP}$-完全问题

- $\mathcal{NP}$-完全问题与$\mathcal{NP}$ - 完全类
  - $\mathcal{NP}$ 类中最“难”的问题称为$\mathcal{NP}$ - 完全（$\mathcal{NP}$ - complete）问题
    - 若某个$\mathcal{NP}$-完全问题有多项式时间算法，则$\mathcal{NP}$类中所有问题均有多项式时间算法
  - $\mathcal{NP}$-完全问题的全体组成$\mathcal{NP}$ - 完全类，记为$\mathcal{NP}$-C
- 归约（reduction）
  - 设有判定问题 Π_1, Π_2，若对 Π_1 的任一实例 I_1，可在多项式时间内构造出 Π_2 的一个实例 I_2，使得 I_1 的答案为“是”当且仅当 I_2 的答案为“是”，则称 Π_1 可多项式时间归约到 Π_2，记为 $\Pi_1 \leq_m^p \Pi_2$
    - 求解 Π_2 不会比求解 Π_1 更容易



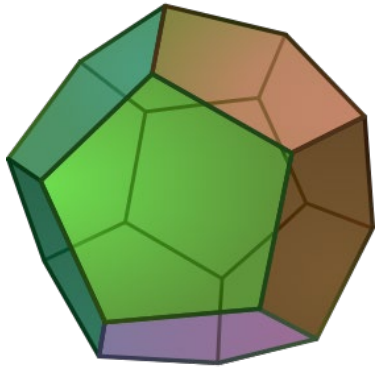
Π_1 的算法

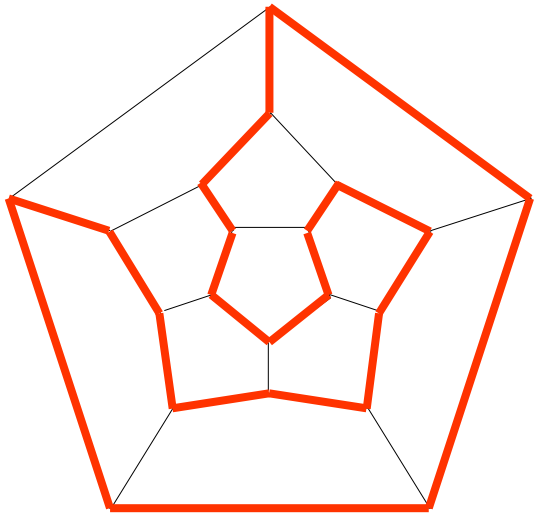
可用求解 Π_2 的多项式时间算法设计出求解 Π_1 的多项式时间算法，反之未必成立

Hamiltion图

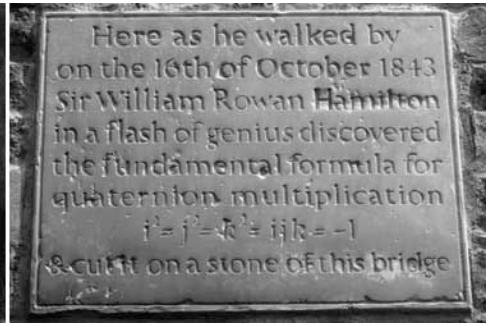
- Hamiltion圈与Hamiltion图
  - 经过图的所有顶点恰好一次的圈称为 Hamilton圈（Hamiltion cycle）
  - 存在Hamilton圈的图称为Hamilton图
  - HC问题，即判断一个图是否为Hamiltion图，是$\mathcal{NP}$-完全问题
- Icosian game（Hamiltion, 1857）
  - 一个正十二面体的二十个顶点各代表一个城市，是否有一条从某个城市出发，沿正十二面体的棱行走，经过每个城市恰好一次，最后回到出发城市的路线

Amsterdam, Ann Arbor, Berlin, Budapest, Dublin, Edinburgh, Jerusalem, London, Melbourne, Moscow, Novosibirsk, New York, Paris, Peking, Prague, Rio di Janeiro, Rome, San Francisco, Tokyo, Warsaw









William Rowan Hamilton
爱尔兰数学家(1805-1865)
爱尔兰都柏林Broom Bridge

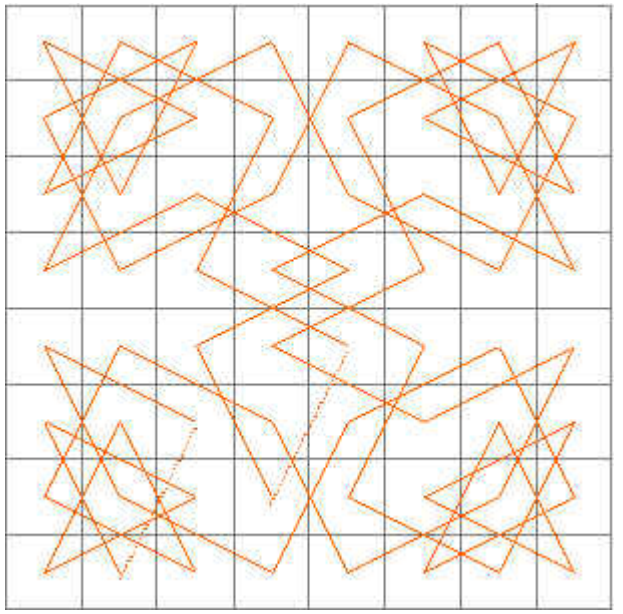
Hamiltion图

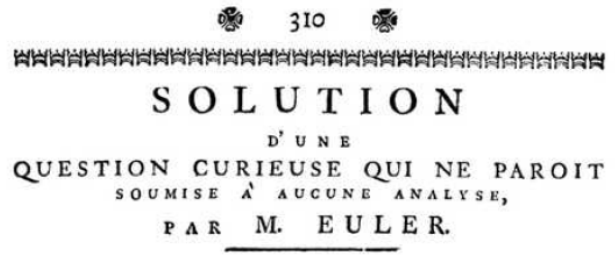


组合优化

MATH T

- 骑士环游（Knight's tour）
  - 在 8×8 国际象棋棋盘上，马能否按其走子规则，从一个格子出发，经过其它格子恰好一次，最后回到起点
    - 构造“跳马图”，每一格子为图的一个顶点，两个格子之间有边相连当且仅当马可按走子规则从一个格子跳到另一个格子
  - $m\times n\ (m\le n)$ 方格棋盘对应的“跳马图”为Hamiltonian图，除非
    - m,n 均为奇数
    - 或 $m=1,2,4$
    - 或 $m=3,n=4,6,8$





310

SOLUTION

D'UNE

QUESTION CURIEUSE QUI NE PAROIT

SOUMISE A AUCUNE ANALYSE,

PAR M. EULER.

Euler L, Solution of a curious question which does not seem to have been subjected to any analysis, *Mémoires de l'Academie Royale des Sciences et Belles Lettres*, 15, 310–337, 1759

Schwenk AJ. Which rectangular chessboards have a knight's tour? *Mathematics Magazine*, 64, 325-332, 1991.

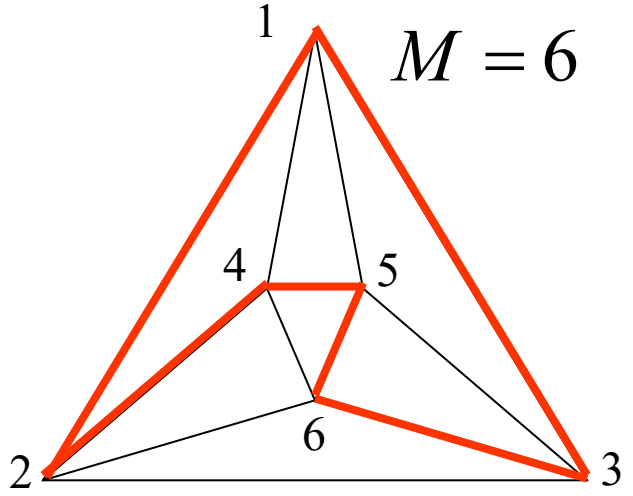
归约



组合优化

MATH T

- $\mathrm{HC} \leq_m^p \mathrm{TSP}$
  - 任给HC问题的实例 I_{HC}：图 $G=(V,E)$
  - 构造TSP判定形式的实例 I_{TSP}
    - 城市数 $n=|V|$，城市与图中顶点一一对应。整数阈值 $M=|V|$
    - 城市间距离 $c_{ij}=\begin{cases}1, & (v_i,v_j)\in E,\\ 2, & (v_i,v_j)\notin E,\end{cases} \quad i\neq j$
  - I_{HC} 的答案为“是”当且仅当 I_{TSP} 的答案也为“是”
    - G 中存在Hamilton圈当且仅当存在总长度不超过$|V|$的环游
      - 若 G 中存在一Hamilton 圈，则按该圈经过各顶点顺序依次到达每个城市，圈中每条边的两个端点所对应的两个城市间距离均为1，环游总长度恰为$|V|$
      - 若环游总长度不超过 $|V|$，由于环游需经过 $|V|$ 个城市，环游中相邻两个城市间距离均为 1，它们所对应的 G 中两个顶点之间均有边相连，所有这些边恰组成 G 的一个Hamilton圈



$$\begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 2 \\ 1 & 0 & 1 & 1 & 2 & 1 \\ 1 & 1 & 0 & 2 & 1 & 1 \\ 1 & 1 & 2 & 0 & 1 & 1 \\ 1 & 2 & 1 & 1 & 0 & 1 \\ 2 & 1 & 1 & 1 & 1 & 0 \end{pmatrix}$$

$\mathcal{NP}$-完全问题



组合
优化
MATHT

- $\mathcal{NP}$-完全问题定义
  - 若 $\Pi \in \mathcal{NP}$，且对任意的 $\Pi' \in \mathcal{NP}, \Pi' \leq_m^p \Pi$ ，则称 Π 是 $\mathcal{NP}$ -完全问题
- $\mathcal{NP}$-完全问题判定定理
  - 若 $\Pi \in \mathcal{NP}$，且存在某个 $\Pi_1 \in \mathcal{NP}\text{-}\mathcal{C}, \Pi_1 \leq_m^p \Pi$ ，则 Π 是 $\mathcal{NP}$ - 完全问题
    - 任取 $\Pi' \in \mathcal{NP}$ ，由于 $\Pi_1 \in \mathcal{NP}\text{-}\mathcal{C}$ ，故由 $\mathcal{NP}$ - 完全问题的定义，$\Pi' \leq_m^p \Pi_1$
    - 由 $\Pi_1 \leq_m^p \Pi$ 和归约的传递性，$\Pi' \leq_m^p \Pi$
    - 由 Π' 的任意性和 $\mathcal{NP}$ - 完全问题的定义，$\Pi \in \mathcal{NP}\text{-}C$

$\mathcal{NP}$-完全问题判定定理简化了 $\mathcal{NP}$-完全问题的证明方法，但运用该定理需要一个已知的 $\mathcal{NP}$-完全问题，意味着第一个 $\mathcal{NP}$-完全问题只能依据定义判别

XXX, 哈密顿图判定问题的多项式时间算法。计算机科学, **47 (7), 8-20, 2020.**



计 算 机 科 学
COMPUTER SCIENCE

计算机科学 COMPUTER SCIENCE
http://www.jsjkx.com
DOI:10.11896/jsjkx.191200176

哈密顿图判定问题的多项式时间算法

湘潭大学计算机学院·网络空间安全学院　湖南 湘潭 411105
国防科技大学计算机学院　长沙 410073
智能计算和信息处理教育部重点实验室　湖南 湘潭 411105

摘　要　NP=?P(即 NP 是否等于 P)的问题是计算机科学和数学中的重要问题。美国克雷数学研究院将其列为新千年七大困难问题之首，2005 年 *Science* 将其列为 25 个困难问题之 19。*Science* 最近列出的 125 个亟待解决的重要问题中，第 19 个问题实质上就是 NP=?P 的问题。如果 NP=P，对于很多困扰科学研究的困难计算问题，理论上就存在多项式时间算法来迅速求解它们。而现代密码学建立在 NP≠P 的假设之上。人们希望存在难解问题，希望基于难解问题构造加密算法，希望能够利用难解问题的求解复杂性对抗分析和攻击。如果 NP=P，所有在 NP≠P 假定之上开展的计算研究都至少需要重新审视其意义。NP 完全问题的求解复杂性决定 NP=P 是否成立。针对一个被称为 MSP 问题的新问题，文中提出了一个关于 MSP 问题的多项式时间算法，并给出了该算法的证明和时间复杂性分析。由于已经发表了十多个经典的 NP 完全问题到 MSP 问题的归结以及 MSP 问题到 SAT 问题的归结，因此 MSP 问题存在多项式时间算法这样一个研究结果对于研究 NP=P 有重要和积极的意义。

关键词：MSP 问题；HC 问题；NP 完全问题；多项式时间算法
中图法分类号　TP301.6

数理逻辑

组合优化
MATH T

- 数理逻辑（mathematical logic）
  - 用数学的方法研究逻辑推理和数学计算，将推理论证、数学计算的过程符号化、形式化、公理化的学科



Kurt Friedrich Gödel (1906 - 1978)
奥地利哲学家、数学家



John von Neumann (1903 - 1957)
匈牙利裔美国科学家

Princeton 20./III.1956.

Lieber Herr v. Neumann!

Ich habe mit grösstem Bedauern von Ihrer Erkrankung gehört. Die Nachricht kam mir ganz unerwartet. Morgenstern hatte mir zwar schon im Sommer von einem Schwächeanfall erzählt, den Sie einmal hatten, aber er meinte damals, dass dem keine grössere Bedeutung beizumessen sei. Wie ich höre, haben Sie sich in den letzten Monaten einer radikalen Behandlung unterzogen u. ich freue mich, dass diese den gewünschten Erfolg hatte u. es Ihnen jetzt besser geht. Ich hoffe u. wünsche Ihnen, dass

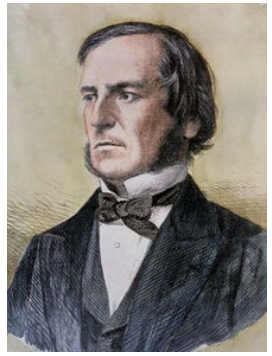
1956年Gödel致von Neumann信，信中对若干数理逻辑问题算法和复杂性的讨论被认为是计算复杂性研究的开端

2006年Gödel奖获奖者: Manindra Agrawal, Neeraj Kayal, Nitin Saxena

Gödel奖由欧洲理论计算机协会 **(European Association for Theoretical Computer Science, EATCS)** 和**ACM**协会算法和计算理论研究组 **(Special Interest Group on Algorithms and Computational Theory, ACM SIGACT)** 联合颁发，奖励理论计算机科学领域卓越论文的作者

Boolean表达式

- Boolean变量
  - 仅可取“真”和“假”（或“1”和“0”）两种值的变量称为**Boolean变量**
  - Boolean变量的运算
    - 非（negation） $\neg x = 1 \Leftrightarrow x = 0$
    - 析取（disjunction） $x_1 \vee x_2 = 1 \Leftrightarrow x_1 = 1$或 $x_2 = 1$
    - 合取（conjunction） $x_1 \wedge x_2 = 1 \Leftrightarrow x_1 = 1$且 $x_2 = 1$
- Boolean表达式
  - 若干**Boolean**变量用运算符和括号按一定的逻辑关系联结起来的表达式称为Boolean表达式（**Boolean expression**）
  - 对出现在**Boolean**表达式中的所有**Boolean**变量各指定一个值，可按**Boolean**变量的运算法则确定表达式的值**1**或**0**



George Boole (1815-1864)
英国数学家、哲学家、逻辑学家



Geoffrey Everest Hinton (1947-)
英裔加拿大计算机学家
2018年Turing奖得主
2024年Nobel物理学奖得主

George Boole → 女 → **Mary Ellen Hinton (1856-1908)** → 子 → **George Boole Hinton (1882-1943)** → 子 → **Howard Everest Hinton (1912-1977)**
昆虫学家、英国皇家学会会士

Mary Ellen Hinton (1856-1908) → 外孙 → **William H. Hinton(韩丁) (1919-2004)**
Mary Ellen Hinton (1856-1908) → 外孙女 → **Joan Chase Hinton(寒春) (1921-2010)** × **Erwin Engst(阳早) (1919-2003)**

合取范式



组合优化
MATH T

- 合取范式（conjunctive normal form, CNF）
  - 文字（**literal**）：变量或变量的非
  - 子句（**clause**）：若干个文字的析取
  - 合取范式：若干个子句的合取
- Boolean表达式的等值演算
  - Boolean变量的运算遵从双重否定律、De Morgan律、析取对合取的分配率、合取对析取的分配率等运算定律
  - 任一Boolean表达式都存在与之等值的合取范式

$$\neg(\neg x_1 \vee x_2) \vee x_3 \Leftrightarrow (\neg\neg x_1 \wedge \neg x_2) \vee x_3 \Leftrightarrow (x_1 \wedge \neg x_2) \vee x_3 \Leftrightarrow (x_1 \vee x_3) \wedge (\neg x_2 \vee x_3)$$

De Morgan律　双重否定律　分配律

子句　文字

$\mathcal{NP}$-完全问题



组合优化

MATH T

- 可满足性问题（Satisfiability，SAT）
  - 给定一合取范式，是否存在其变量的一种赋值，使得该表达式值为真

$(x_1 \vee x_3) \wedge (\neg x_2 \vee x_3)$ 是

$(x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge \neg x_1$ 否

- Cook-Levin定理：SAT $\in \mathcal{NP}$-C
  - Cook运用图灵机语言，通过$\mathcal{NP}$问题的一种等价定义，证明了SAT问题的$\mathcal{NP}$-完全性

SAT问题被认为是第一个$\mathcal{NP}$ 一完全问题。

苏联数学家Levin在相近时间内独立证明了包括SAT问题在内的若干问题的$\mathcal{NP}$-完全性



Stephen Arthur Cook
(1939 -)
美国计算机学家
1982年图灵奖得主



Leonid Anatolievich Levin
(1948 -)
苏联计算机学家

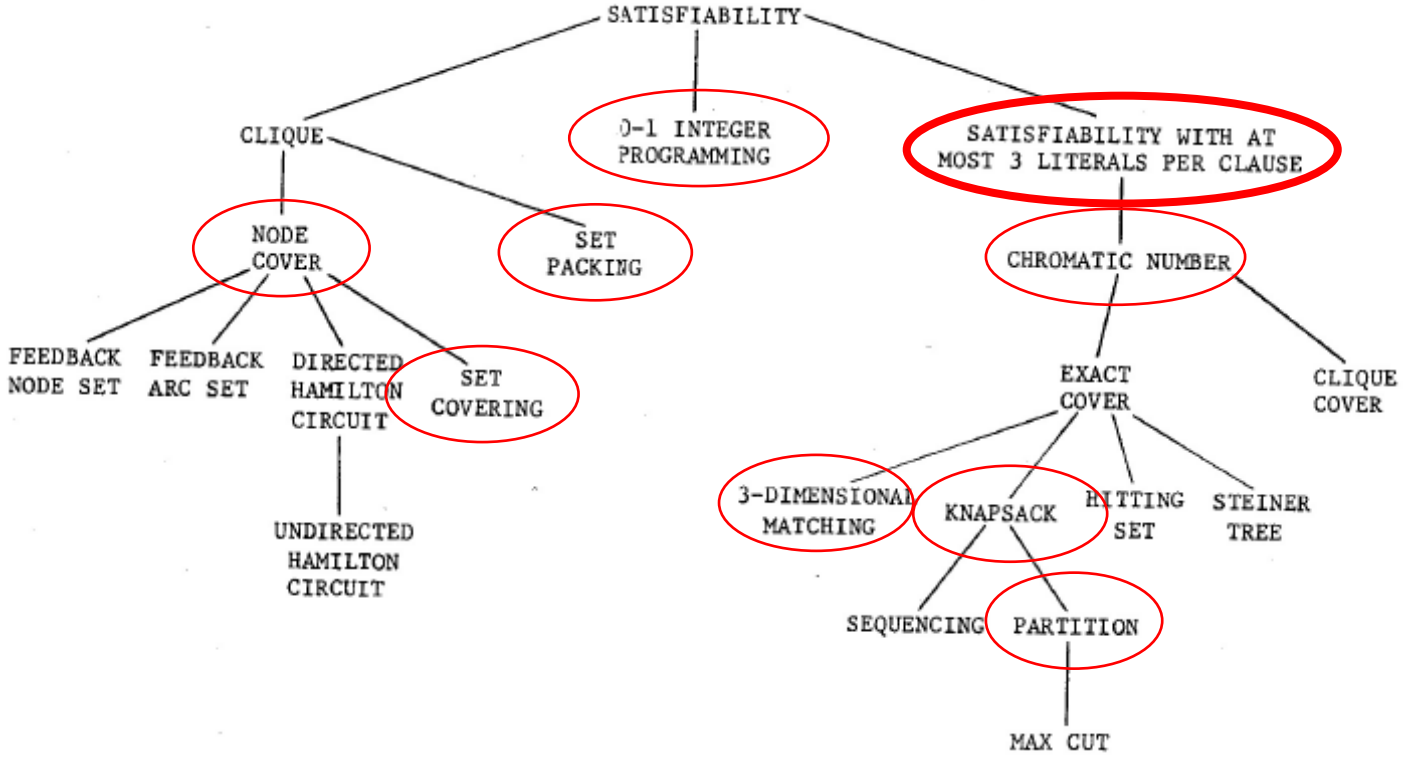
Cook SA. The complexity of theorem-proving procedures, *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing*, 151-158, 1971.

Levin LA, Universal Sequential Search Problems, *Problems of Information Transmission*, 9, 115–116, 1973

王浩（Wang Hao）（1921-1995），美籍华裔数学家、逻辑学家、计算机科学家、哲学家。先后就读于西南联合大学数学系，清华大学研究生院哲学部和哈佛大学。1961-1967任哈佛大学教授，Stephen Cook的博士论文导师

$\mathcal{NP}$-完全问题

- 组合优化问题的$\mathcal{NP}$-完全性
  - 1972年，Karp从SAT问题出发构造归约，证明了20个重要组合优化问题的$\mathcal{NP}$-完全性





Richard Manning Karp
(1935 -)
美国计算机学家
1985年图灵奖得主

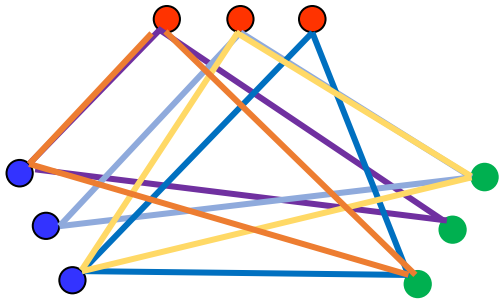
Karp RM. Reducibility among combinatorial problems, *Proceedings of a Symposium on the Complexity of Computer Computations*, 85-103, 1972

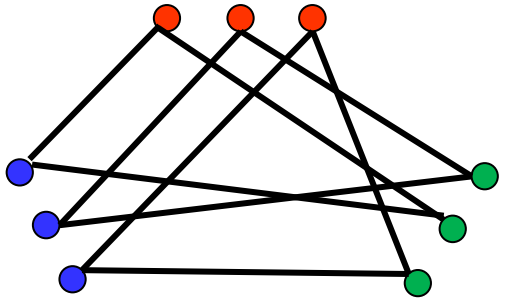
Steve Cook was primarily in math but also in the new CS Department. It is to our everlasting shame that we were unable to persuade the math department to give him tenure. Perhaps they would have done so if he had published his proof of the NP-completeness of satisfiability a little earlier.

——Karp R, A Personal View of Computer Science at Berkeley

三维匹配

- 三维匹配（three-dimensional matching，3DM）
  - 给定三个均含 n 个元素，且两两不相交的集合 $X=\{x_1,\cdots,x_n\}$，$Y=\{y_1,\cdots,y_n\}$ 和 $Z=\{z_1,\cdots,z_n\}$。三元组集合 $T\subseteq X\times Y\times Z$
  - 是否存在 $M\subseteq T$，使得 $X\cup Y\cup Z$ 中任一元素在 M 的三元组中恰好出现一次
- 复杂性
  - 三维匹配是$\mathcal{NP}$-完全问题





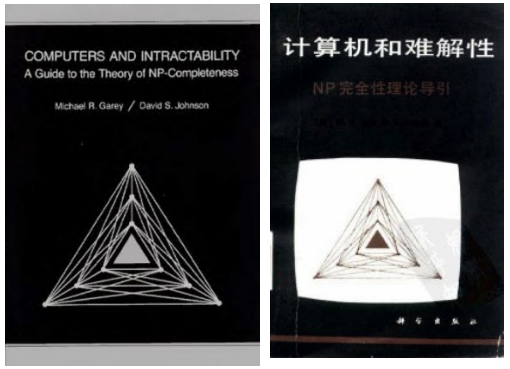
多项式可解	$\mathcal{NP}$-难
2SAT	3SAT
最大基数匹配	三维匹配
指派	三维指派
最短路	最长路
最小割	最大割
最小边覆盖	最小顶点覆盖
Euler圈	Hamiltion圈
中国邮递员	旅行售货商

最小生成树	最小顶点着色
最大生成树	最小边着色

$\mathcal{NP}$-完全问题

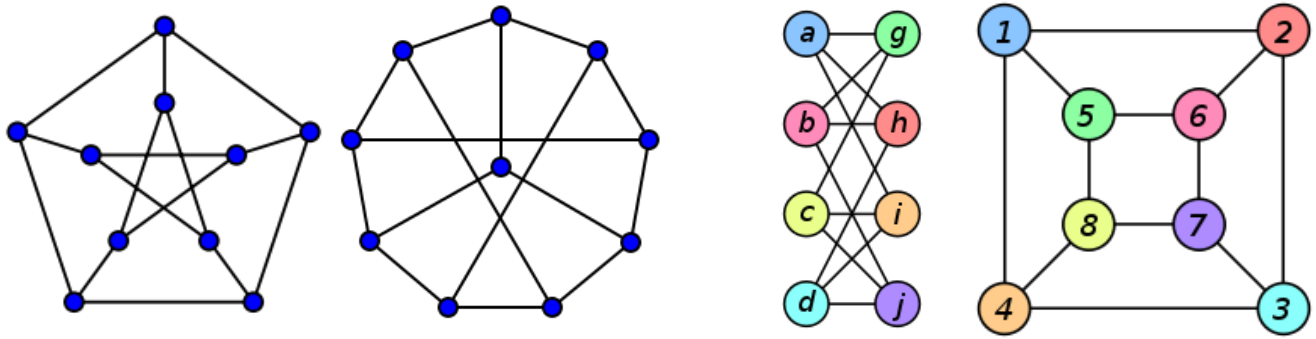
- 三个$\mathcal{NP}$ 问题
  - 线性规划 $\mathcal{P}$
  - 素性测试 $\mathcal{P}$
  - 图同构 ?





Garey MR, Johnson DS. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, 1979.（中译本：计算机和难解性： $\mathcal{NP}$ 完全性理论导引. 张立昂, 沈泓译，科学出版社, 1990.）

- 图同构
  - 称图 $G=(V,E)$ 与图 $G'=(V',E')$ 同构（isomorphic），若存在双射 $\sigma: V \to V'$，使得 G 中两顶点 u,v 相邻（adjacent）当且仅当 G' 中两顶点 $\sigma(u),\sigma(v)$ 相邻
  - 图同构问题（Graph Isomorphism, GI）：给定图 G 与图 G'，判断 G 与 G' 是否同构



Babai L, Luks EM, Canonical labeling of graphs, *Proceedings of the 15th Annual ACM Symposium on Theory of Computing*, 171-183, 1983.

$O\left(2^{\sqrt{n\log n}}\right) \to O\left(2^{\log^c n}\right)$?

$\mathcal{NP}$-完全



组合优化

MATH T

- 子图同构（Subgraph Isomorphism）
  - 给定图 G 与图 G'，判断 G 是否存在一子图 H 与 G' 同构
    - 子图同构问题是 $\mathcal{NP}$ -完全的
      - 设 G 有 n 个顶点，取 G' 为含 n 个顶点的圈，G 是否存在一子图与 G' 同构等价于 G 是否存在一Hamiltonian圈



4. 计算的复杂性

计算的复杂性测度函数有三类：一类是指数型的，常写为 c^n 的形式（c 是常数）；一类是多项式型的，常写为 n^k 的形式（k 为非负整数）；另一类是对数型的，常写为 $\log n$ 的形式. 具有这些复杂性的问题分别称为指数复杂性，多项式复杂性和对数复杂性.

人们习惯于把理论上可计算的问题类称为能行可计算的，而把具有多项式复杂性的问题类称为有效可计算的，通常称为 P 问题. NP 问题是指还未找到多项式复杂性算法的问题.

研究和实验表明，单纯靠提高计算机速度并不能解决 NP 问题. 例如，根据某些计算机的运行记录，对于复杂性为 2^n 的问题，即使计算机速度提高 1 000 倍，也只能是多算约 10 道题. 于是，人们认为，解决 NP 问题的关键是要从数学上找出好的算法. 事实上，数学家们也找到了各种各样的好办法，大大简化了计算. 例如，用计算机对卫星照片进行处理，如果在一张 10 cm^2 的照片上以一微米为间隙打上格子，则处理一张照片需要进行 10^{16} 次运算，即使用每秒百亿次的计算机也要连续算上十多个昼夜. 后来，有人发明了一种好的算法，大大降低了计算的复杂性，使得用同样的计算机计算只需要 $\frac{1}{3}$ 秒.

大学博士研究生学位论文

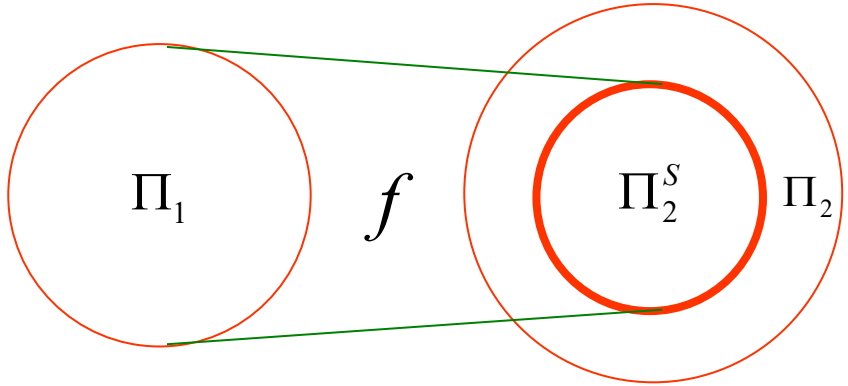
第八章

机器可读的SLA，是一个现实世界合同在计算机空间的映射，可以是任意复杂的形式。它会包括条件，以及并行运行的要求，影响最终价格的可选项目，惩罚，报酬等等。在没有预先知道它们的结构前提下处理这种文档（甚至是良好的定义了语法并且被理解的情况下）是一个十分困难的任务。更准确的说，这个问题等同于图的同构问题，由于其复杂度，仍然是个未解决问题。它的子问题，子图的同构问题即所谓的NP完全问题[69]。

$\mathcal{NP}$-完全性证明

组合优化 MATHT

- $\mathcal{NP}$-完全性证明方法
  - 证明 $\Pi \in \mathcal{NP}$
  - 寻找与 Π 联系紧密，且实例结构较为规范、简明的已知$\mathcal{NP}$ - 完全问题 Π_1
  - 构造归约 $\Pi_1 \leq_m^p \Pi$
    - 任取 Π 的实例 I，构造 Π_1 的实例 I_1
    - 若 I 的答案为“是”，I_1 的答案也为“是”
    - 若 I_1 的答案为“是”，I 的答案也为“是”
      (或：若 I 的答案为“否”，I_1 的答案也为“否”)



- 3SAT: 实例中任一子句包含文字数不超过3
  - 任取3SAT问题的一实例 $(l_{11} \vee l_{12} \vee l_{13}) \wedge (l_{21} \vee l_{22} \vee l_{23}) \wedge \cdots \wedge (l_{k1} \vee l_{k2} \vee l_{k3})$
  - 任取SAT问题的一实例 $(l_{11} \vee l_{12} \vee \cdots \vee l_{1i_1}) \wedge (l_{21} \vee l_{22} \vee \cdots \vee l_{2i_2}) \wedge \cdots \wedge (l_{k1} \vee l_{k2} \vee \cdots \vee l_{ki_k})$
- $2\text{SAT} \in \mathcal{P}$

$$2\text{SAT} \succ 3\text{SAT} \succ 4\text{SAT} \succ \text{SAT}$$

Krom MR. The Decision Problem for a Class of First-Order Formulas in which all Disjunctions are Binary. *Mathematical Logic Quarterly*, 13, 15-20, 1967.

3SAT



组合优化
MATHT

- $SAT \leq_m^p 3SAT$
  - 任取SAT问题一实例 $F_{SAT} = c_1 \wedge c_2 \wedge \cdots \wedge c_m$，构造3SAT问题的实例 F_{3SAT}
    - 若 c_i 中所含文字数不超过3，则 F_{3SAT} 也包含子句 c_i
    - 若 $c_i = l_{i_1} \vee l_{i_2} \vee \cdots \vee l_{i_k}$，则令

      $$D_i = (l_{i_1} \vee l_{i_2} \vee y_{i1}) \wedge (\neg y_{i1} \vee l_{i_3} \vee y_{i2}) \wedge (\neg y_{i2} \vee l_{i_4} \vee y_{i3}) \wedge \cdots \wedge (\neg y_{i,k-4} \vee l_{i_{k-2}} \vee y_{i,k-3}) \wedge (\neg y_{i,k-3} \vee l_{i_{k-1}} \vee l_{i_k})$$

      其中 $y_{i1}, y_{i2} \cdots y_{i,k-3}$ 是新变量且不出现在别处。F_{3SAT} 中包含所有 D_i 中的全部子句
  - 若 F_{SAT} 答案为“是”，F_{3SAT} 答案也为“是”
    - 存在一种赋值，使得 F_{SAT} 中任一子句 c_i 均为真，c_i 中文字至少有一个为真，不妨设为 l_{i_j}
    - 存在一种赋值，使得 F_{3SAT} 中每个子句均为真
      - F_{SAT} 中原有变量赋值保持不变，F_{3SAT} 中包含的 F_{SAT} 中子句均为真
      - F_{3SAT} 中所有 D_i 中的全部子句均为真

      $$c_i = l_{i_1} \vee l_{i_2} \vee \cdots \vee \overset{1}{l_{i_j}} \vee \cdots \vee l_{i_k}$$

      $$D_i = (l_{i_1} \vee l_{i_2} \vee \overset{1}{y_{i1}}) \wedge (\overset{0}{\neg y_{i1}} \vee l_{i_3} \vee \overset{1}{y_{i2}}) \wedge \cdots \wedge (\overset{0}{\neg y_{i,j-4}} \vee l_{i_{j-2}} \vee \overset{1}{y_{i,j-3}}) \wedge (\overset{0}{\neg y_{i,j-3}} \vee l_{i_{j-1}} \vee \overset{1}{y_{i,j-2}}) \wedge (\overset{0}{\neg y_{i,j-2}} \vee \overset{1}{l_{i_j}} \vee \overset{0}{y_{i,j-1}})$$
      $$\wedge (\overset{1}{\neg y_{i,j-1}} \vee l_{i_{j+1}} \vee \overset{0}{y_{i,j}}) \wedge (\overset{1}{\neg y_{i,j}} \vee l_{i_{j+2}} \vee \overset{0}{y_{i,j+1}}) \wedge \cdots \wedge (\overset{1}{\neg y_{i,k-4}} \vee l_{i_{k-2}} \vee \overset{0}{y_{i,k-3}}) \wedge (\overset{1}{\neg y_{i,k-3}} \vee l_{i_{k-1}} \vee l_{i_k})$$

3SAT



组合优化

MATHT

- $SAT \leq_m^p 3SAT$
  - 任取SAT问题一实例 $F_{SAT} = c_1 \wedge c_2 \wedge \cdots \wedge c_m$，构造3SAT问题的实例 F_{3SAT}
    - 若 c_i 中所含文字数不超过3，则 F_{3SAT} 也包含子句 c_i
    - 若 $c_i = l_{i_1} \vee l_{i_2} \vee \cdots \vee l_{i_k}$，则令

      $$D_i = (l_{i_1} \vee l_{i_2} \vee y_{i1}) \wedge (\neg y_{i1} \vee l_{i_3} \vee y_{i2}) \wedge (\neg y_{i2} \vee l_{i_4} \vee y_{i3}) \wedge \cdots \wedge (\neg y_{i,k-4} \vee l_{i_{k-2}} \vee y_{i,k-3}) \wedge (\neg y_{i,k-3} \vee l_{i_{k-1}} \vee l_{i_k})$$

      其中 $y_{i1}, y_{i2} \cdots y_{i,k-3}$ 是新变量且不出现在别处。F_{3SAT} 中包含所有 D_i 中的全部子句
  - 若 F_{3SAT} 答案为“是”，F_{SAT} 答案也为“是”
    - 存在一种赋值，使得 F_{3SAT} 中任一子句均为真
    - 将该赋值限制到 F_{SAT} 中的变量，得到 F_{SAT} 的一种赋值。在该赋值下，F_{SAT} 中任一子句均为真
      - 若 F_{SAT} 中某个子句 c_i 为假，c_i 中的每个文字均为假
      - F_{3SAT} 中子句 D_i 也为假，矛盾

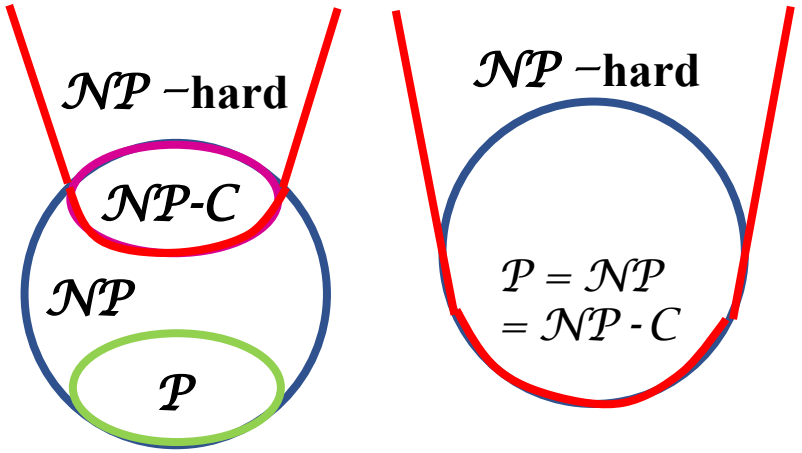
        $$c_i = \overset{0}{l_{i_1}} \vee \overset{0}{l_{i_2}} \vee \cdots \vee \overset{0}{l_{i_j}} \vee \cdots \vee \overset{0}{l_{i_k}}$$

        $$D_i = (\overset{0}{l_{i_1}} \vee \overset{0}{l_{i_2}} \vee \overset{1}{y_{i1}}) \wedge (\overset{0}{\neg y_{i1}} \vee \overset{0}{l_{i_3}} \vee \overset{1}{y_{i2}}) \wedge (\overset{0}{\neg y_{i2}} \vee \overset{0}{l_{i_4}} \vee \overset{1}{y_{i3}}) \wedge \cdots$$

        $$\wedge (\overset{0}{\neg y_{i,k-5}} \vee \overset{0}{l_{i_{k-3}}} \vee \overset{1}{y_{i,k-4}}) \wedge (\overset{0}{\neg y_{i,k-4}} \vee \overset{0}{l_{i_{k-2}}} \vee \overset{1}{y_{i,k-3}}) \wedge (\overset{0}{\neg y_{i,k-3}} \vee \overset{0}{l_{i_{k-1}}} \vee \overset{0}{l_{i_k}})$$

$\mathcal{NP}$-难问题

- $\mathcal{NP}$-难问题
  - $\mathcal{NP}$ -难（$\mathcal{NP}$ -hard）问题至少和$\mathcal{NP}$ -完全问题一样难
  - 若一优化问题的判定形式是$\mathcal{NP}$ -完全的，则该优化问题是$\mathcal{NP}$ -难的
- $\mathcal{P}$，$\mathcal{NP}$，$\mathcal{NP}$-完全与$\mathcal{NP}$-难
  - $\mathcal{P}$ 类与$\mathcal{NP}$-完全类是$\mathcal{NP}$ 类的两个子集。在$\mathcal{P} \neq \mathcal{NP}$ 的假设下，两者交为空，且两者并不等于$\mathcal{NP}$ 。若$\mathcal{P} = \mathcal{NP}$，两者相等
  - 在$\mathcal{P} \neq \mathcal{NP}$ 假设下，$\mathcal{NP}$-完全问题与$\mathcal{NP}$ -难问题均没有多项式时间算法。即使$\mathcal{P} = \mathcal{NP}$，部分$\mathcal{NP}$ -难问题仍没有多项式时间算法



$\mathcal{P} \neq \mathcal{NP}$　　$\mathcal{P} = \mathcal{NP}$

$\mathcal{NP}$ 问题是~~没有~~多项式时间算法的问题

背包问题是一个$\mathcal{NP}$ 问题，~~因此~~难解

$\mathcal{NP}$ - 完全问题是~~所有~~问题中最难的问题

思考题

- 3SAT变形的 $\mathcal{NP}$-完全性证明
  - 3SAT：每个子句恰有3个不同的变量（文字）
  - NAE3SAT（Non All Equal）：是否存在一真值赋值，每个子句的文字有不同的取值
  - 1-in-3SAT（One-in-three-3SAT）：是否存在一真值赋值，每个子句中恰有一个文字取真
  - 3,4-SAT：每个子句恰有3个不同的变量，每个变量在所有子句中出现不超过4次
  - $\left(3^{n-1},\infty^{1}\right)$-SAT：每个子句恰有3个不同的变量，除一个变量外，其余每个变量在所有子句中恰出现3次

Darmann A, Döcker J, On a simple hard variant of Not-All-Equal 3-Sat, *Theoretical Computer Science*, 815, 147-152, 2020.

子问题



组合优化

MATH T

- 子问题
  - 在问题Π的实例构成上增加一些限制形成Π的一个子问题 (subproblem) Π'
    - Π'的实例集包含在Π的实例集中，Π'的答案为“是”（“否”）的实例集恰为Π的答案为“是”（“否”）的实例集与Π'的实例集之交
  - 子问题的计算复杂性
    - 若 $\Pi \in \mathcal{P}$，则 $\Pi' \in \mathcal{P}$，但反之不然
    - 若Π'是$\mathcal{NP}$-完全的，则Π也是$\mathcal{NP}$-完全的，但反之不然
    - 目标是寻找“最特殊”的$\mathcal{NP}$-完全子问题和“最一般”的$\mathcal{P}$子问题

SAT

3SAT

NAE3SAT　　3,4-SAT

1-in-3SAT

$\left(3^{n-1},\infty^1\right)$-SAT

$\left(3^{n-3},4^3\right)$-SAT

3,3-SAT

2SAT

子问题

- 子集和、划分与背包
  - 划分问题（Partitioning）
    - 给定正整数集$A=\{a_1,a_2,\cdots,a_n\}$，问是否存在集合A_1,A_2，使得$A=A_1\cup A_2$，$A_1\cap A_2=\phi$，且$\sum_{a_j\in A_1}a_j=\sum_{a_j\in A_2}a_j$
  - 划分问题是子集和问题当$B=\frac{1}{2}\sum_{j=1}^{n}a_j$时的子问题
  - 子集和问题是背包问题判定形式物品 j 的价值与大小均为 a_j，容量和阈值均为 B 时的子问题
  - 子集和问题的优化形式
    - 极大化：求子集A_1，使得$\sum_{a_i\in A_1}a_i\le B$且$\sum_{a_i\in A_1}a_i$尽可能大
    - 极小化：求子集A_1，使得$\sum_{a_i\in A_1}a_i\ge B$且$\sum_{a_i\in A_1}a_i$尽可能小

子集和问题：给定正整数集 $A=\{a_1,a_2,\cdots,a_n\}$和数 B，问是否存在子集 $A_1\subseteq A$，使得 $\sum_{a_i\in A_1}a_i=B$

背包问题判定形式：n 个物品，物品 j 的价值为 p_j，大小为 w_j，背包容量为 C。问是否存在物品集的一个子集，使得子集中物品大小之和不超过背包容量，价值之和不小于M

$$\sum_{a_i\in A_1}a_i\le B,\sum_{a_i\in A_1}a_i\ge B$$

强$\mathcal{NP}$-完全



组合优化

MATH T

- 强$\mathcal{NP}$-完全、普通意义下的$\mathcal{NP}$-完全、强$\mathcal{NP}$-难
  - 设Π为一$\mathcal{NP}$-完全问题，且存在多项式函数p，使得Π的某个所有实例满足$m(I) \le p(s(I))$的子问题Π'是$\mathcal{NP}$-完全的，则称Π为强$\mathcal{NP}$-完全（strongly $\mathcal{NP}$-complete）问题
  - 在$\mathcal{P} \neq \mathcal{NP}$的假设下，任何强$\mathcal{NP}$-完全问题不存在伪多项式时间算法
    - 反证：若Π存在伪多项式时间算法A，时间复杂度为$f(I) = O(\text{poly}(s(I), m(I)))$
    - 用A求解Π的子问题Π'，时间复杂度为
      $$f(I) = O(\text{poly}(s(I), m(I))) = O(\text{poly}(s(I), p(\text{s}(I)))) = O(\text{poly}(s(I)))$$
    - A为Π'的多项式时间算法，与Π'的$\mathcal{NP}$-完全性矛盾
  - 若Π为$\mathcal{NP}$-完全问题，且存在伪多项式时间算法，则称Π为普通意义下的$\mathcal{NP}$-完全问题（$\mathcal{NP}$-complete in the ordinary sense）
  - 至少和强$\mathcal{NP}$-完全问题一样难的问题称为强$\mathcal{NP}$-难（strongly $\mathcal{NP}$-hard）问题
    - TSP为强$\mathcal{NP}$-难问题，背包为普通意义下的$\mathcal{NP}$-难问题

强$\mathcal{NP}$-完全



组合优化
MATH T

- 强$\mathcal{NP}$-完全问题的证明
  - 问题是$\mathcal{NP}$-完全的，且其所有实例均满足 $m(I) \le \text{poly}(s(I))$
    - Hamilton图问题、SAT问题等
  - 归约证明某问题的$\mathcal{NP}$-完全性时所构造的该问题的所有实例均满足 $m(I) \le \text{poly}(s(I))$
    - TSP问题
  - 按强$\mathcal{NP}$-完全问题的定义证明
    - 寻找多项式函数 p 和该问题的一个所有实例满足 $m(I) \le p(s(I))$ 的子问题，通过归约证明该子问题是$\mathcal{NP}$-完全的
  - 用伪多项式时间归约从一个已知强$\mathcal{NP}$-完全问题出发证明某问题的强$\mathcal{NP}$-完全性

伪多项式时间归约

- 伪多项式时间归约
  - 设有判定问题 Π_1, Π_2，若对 Π_1 的任一实例 I_1，可在 $O(\text{poly}(s(I_1), m(I_1)))$ 时间内构造出 Π_2 的一个实例 $I_2 = f(I_1)$，使得
    - I_1 的答案为“是”当且仅当 I_2 的答案也为“是”
    - $m(I_2) \le p_1(s(I_1), m(I_1))$，这里 p_1 为某个二元多项式
    - $s(I_1) \le p_2(s(I_2))$，这里 p_2 为某个多项式

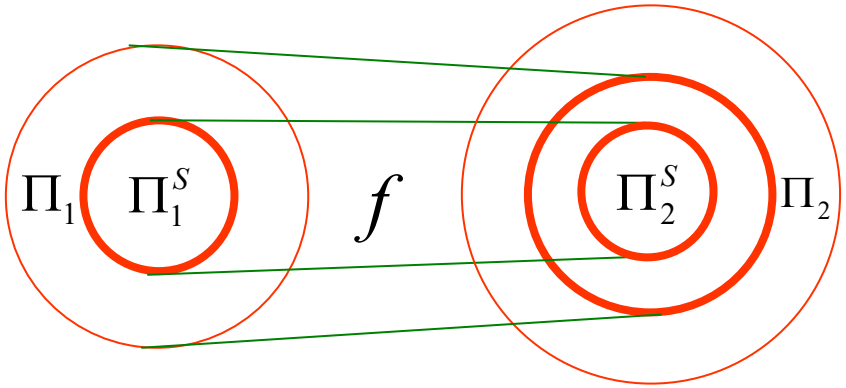
    则称 Π_1 可伪多项式时间归约到 Π_2，记为 $\Pi_1 \le_m^{pp} \Pi_2$
  - 若 Π_1 为强 $\mathcal{NP}$ −完全问题，$\Pi_2 \in \mathcal{NP}$ 且 $\Pi_1 \le_m^{pp} \Pi_2$，则 Π_2 也为强 $\mathcal{NP}$ −完全问题
    - 存在 Π_1 的子问题 Π_1^S，其实例集 $\mathcal{I}_1$ 中所有实例 I_1 满足 $m(I_1) \le p(s(I_1))$，且 Π_1^S 是 $\mathcal{NP}$ −完全的
    - 以 $\mathcal{I}_2 = f(\mathcal{I}_1)$ 为实例集的 Π_2 的子问题为 Π_2^S
      - Π_2^S 的所有实例 I_2 满足 $m(I_2) \le p'(s(I_2))$
    - f 给出了 Π_1^S 到 Π_2^S 的多项式时间归约，即 $\Pi_1^S \le_m^p \Pi_2^S$
    - Π_2^S 是 $\mathcal{NP}$ −完全问题，故 Π_2 是强 $\mathcal{NP}$ −完全问题

$$O(\text{poly}(s(I_1), m(I_1))) = O(\text{poly}(s(I_1)))$$

$$\begin{aligned} m(I_2) &\le p_1(s(I_1), m(I_1)) \\ &\le p_1(s(I_1), p(s(I_1))) = p_1'(s(I_1)) \\ &\le p_1'(p_2(s(I_2))) = p'(s(I_2)) \end{aligned}$$

$$s(I_1) = \Omega(2^{s(I_2)}) \Rightarrow s(I_2) = O(\log(s(I_1)))$$

$$\not\Rightarrow m(I_2) \le p'(s(I_2))$$

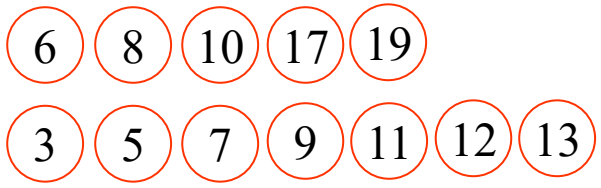


实例构造的时间不太长，要求弱于多项式时间归约

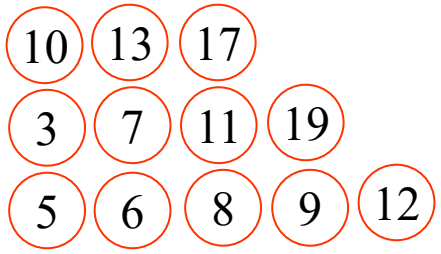
最大数没有显著增大　　实例规模没有显著缩小

3-划分问题

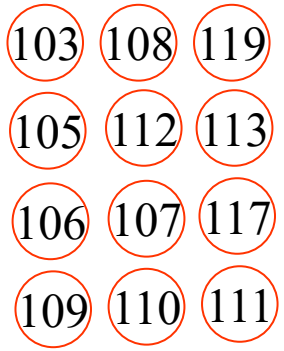
- 3-划分（3-partioning）
  - 给定由 $3m$ 个正整数组成的集合 $A=\{a_1,a_2,\cdots,a_{3m}\}$和正整数 B ，其中 $\frac{B}{4}<a_j<\frac{B}{2}, j=1,2,\cdots,3m$ ，且 $\sum_{j=1}^{3m}a_j=mB$。问是否可将 A 划分成 m 个互不相交的集合 $A_1,A_2,\cdots,A_m$ ，使得对任意的 $i,1\le i\le m,\sum_{a_j\in A_i}a_j=B$
- 3-划分为强$\mathcal{NP}$ −完全问题



划分



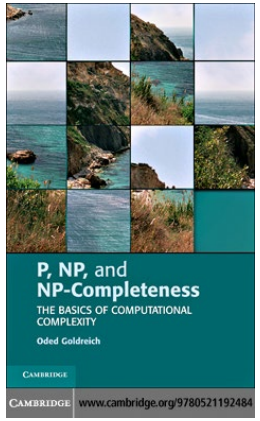
划分

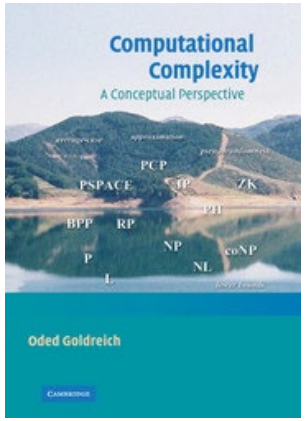


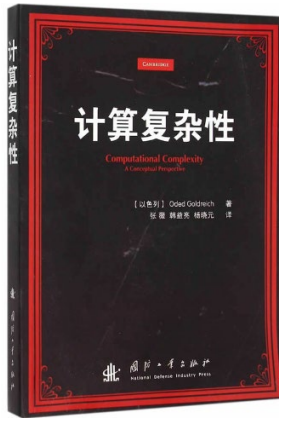
3-划分

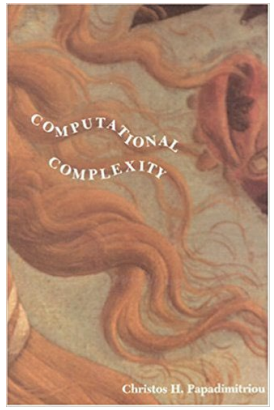
参考资料

组合优化
MATHT





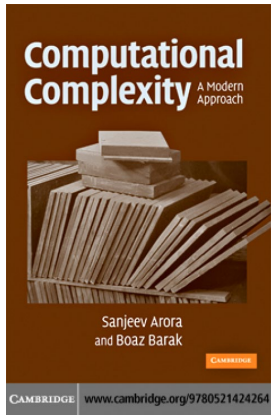


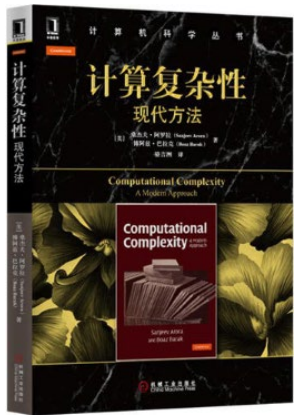


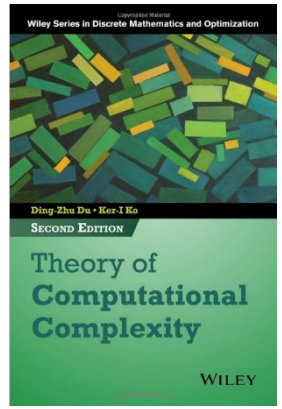
Goldreich O. *P, NP, and NP-Completeness: The basics of Computational Complexity*. Cambridge University Press, 2010.

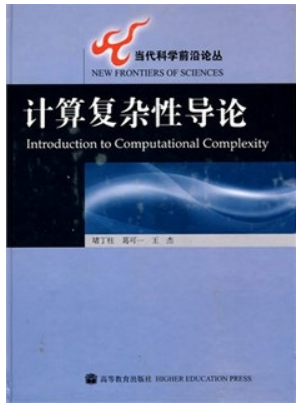
Goldreich O. *Computational Complexity: A Conceptual Perspective*. Cambridge University Press, 2008.（中译本：张薇、韩益亮、杨晓元译，计算复杂性，国防工业出版社，2015）

Papadimitriou CH, *Computational Complexity*, Pearson, 1993









Arora S, Barak B. Computational Complexity: A Modern Approach. Cambridge University Press, 2009.（中译本：计算复杂性：现代方法. 骆吉洲译，机械工业出版社，2015.）

Du DZ, Ko KI. *Theory of Computational Complexity*. Wiley, 2011.

堵丁柱，葛可一，王杰，计算复杂性导论，高等教育出版社，2002.

NP 优化问题汇编



组合优化

MATH T

- **A compendium of NP optimization problems (https://www.csc.kth.se/~viggo/problemlist/compendium.html)**

- Introduction
- Graph Theory
  - Covering and Partitioning
  - Subgraphs and Supergraphs
  - Vertex Ordering
  - Iso- and Other Morphisms
  - Miscellaneous
- Network Design
  - Spanning Trees
  - Cuts and Connectivity
  - Routing Problems
  - Flow Problems
  - Miscellaneous
- Sets and Partitions
  - Covering, Hitting, and Splitting
  - Weighted Set Problems
- Storage and Retrieval
  - Data Storage
  - Compression and Representation
  - Miscellaneous
- Sequencing and Scheduling
  - Sequencing on One Processor
  - Multiprocessor Scheduling
  - Shop Scheduling
  - Miscellaneous
- Mathematical Programming
- Algebra and Number Theory
  - Solvability of Equations
- Games and Puzzles
- Logic
  - Propositional Logic
  - Miscellaneous
- Automata and Language Theory
  - Automata Theory
  - Formal Languages
  - Miscellaneous
- Program Optimization
  - Code Generation
- Miscellaneous
- Bibliography
- Index
- About this document ...

MINIMUM METRIC TRAVELING SALESPERSON PROBLEM

- INSTANCE: Set C of m cities, distances $d(c_i, c_j) \in N$ satisfying the triangle inequality.
- SOLUTION: A tour of C, i.e., a permutation $\pi : [1..m] \rightarrow [1..m]$.
- MEASURE: The length of the tour.
- *Good News:* Approximable within 3/2 [114].
- *Bad News:* APX-complete [394].

浙江大学
ZHEJIANG UNIVERSITY

线性规划和整数规划简介



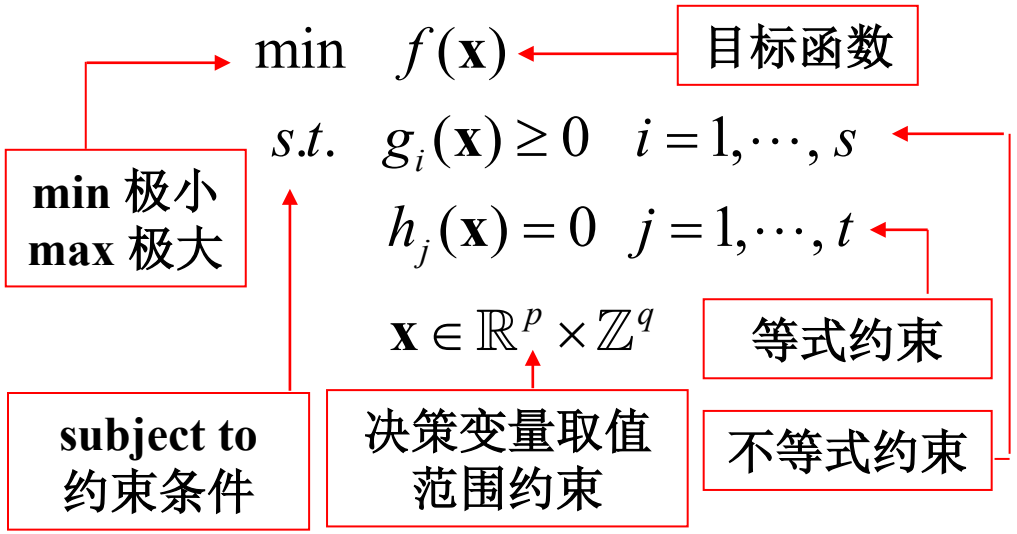
数学规划



组合优化

MATHT

- 数学规划
  - 若干个变量在满足一些等式或不等式限制条件下，使目标函数取得最大值或最小值



- 可行域 $S = \{\mathbf{x} \in \mathbb{F}^n \mid g_i(\mathbf{x}) \geq 0, i = 1, \cdots, s, h_j(\mathbf{x}) = 0, j = 1, \cdots, t\}$
- 可行解 $\mathbf{x} \in S$
- 最优解
  - $\mathbf{x}^* \in S$ ，且对任意 $\mathbf{x} \in S, f(\mathbf{x}^*) \leq f(\mathbf{x})$
- 最优值 $f(\mathbf{x}^*)$

数学规划分类



组合优化

MATH T

- 数学规划分类
  - 线性规划（linear programming）
    - 目标函数为线性函数，约束条件为线性等式或不等式
  - 整数规划（integer programming）：至少有一个决策变量限定取整数值
    - 整数决策变量意义
      - 用于表示只能取离散值的对象的数量
      - 用于表示约束条件之间的逻辑关系或复杂的函数形式
      - 用于表示非数值的优化或可行性问题
    - 特殊整数规划
      - 部分决策变量取整数值的数学规划特称为混合整数规划（Mixed Integer Programming, MIP）
      - 0-1规划：决策变量仅取值0或1的数学规划
      - 整数线性规划：有决策变量取整数值的线性规划

$$\begin{aligned} \min \quad & f(\mathbf{x}) \\ s.t. \quad & g_i(\mathbf{x}) \geq 0 \quad i = 1, \cdots, s \\ & h_j(\mathbf{x}) = 0 \quad j = 1, \cdots, t \\ & \mathbf{x} \in \mathbb{R}^n \end{aligned}$$

$$\begin{aligned} \min \quad & \sum_{i=1}^{n} c_i x_i \\ s.t. \quad & \sum_{j=1}^{n} a_{ij} x_j = b_i, i = 1, \cdots, m \\ & x_1, \cdots, x_n \geq 0 \end{aligned}$$

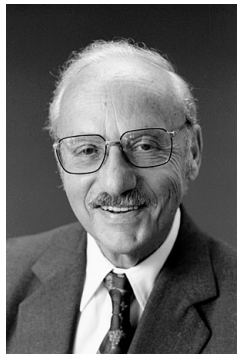
$$\begin{aligned} \min \quad & \mathbf{cx} \\ s.t. \quad & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

$$\begin{aligned} \min \quad & \mathbf{cx} \\ s.t. \quad & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{x} \in \mathbb{R}^p \times \mathbb{Z}^q \end{aligned}$$

线性规划

- 线性规划的求解算法
  - 单纯形法（Simplex Method）
    - 1947年Dantzig提出了求解线性规划的单纯形法
    - 单纯形法是指数时间算法。存在含 m 个变量 m 个约束的线性规划，单纯形法需要进行 $2^m - 1$ 次迭代
      - 实践表明，对多数线性规划问题，单纯形法迭代次数为变量和约束数的多项式函数
  - 多项式时间算法
    - 1979年，Khachiyan 给出了求解线性规划的第一个多项式时间算法椭球算法（Ellipsoid algorithm）
    - 1984年，Karmarkar 给出了实际效果更好的多项式时间算法内点法（Interior Point Method），产生了深远的影响

Klee V, Minty GJ, How good is the simplex algorithm? In Inequalities – III (Shisha O, Eds.), Academic Press, 159–175, 1972



George Bernard Dantzig (1914-2005)
美国数学家



Leonid Genrikhovich Khachiyan (1952-2005)
苏联数学家



Narendra Karmarkar (1957-)
印度数学家

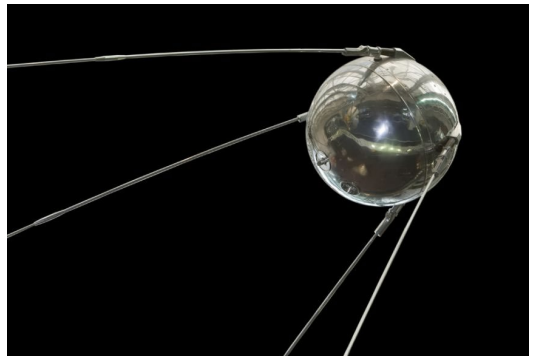
Khachiyan L, A polynomial algorithm in linear programming. *Doklady Akademiia Nauk SSSR*, 244, 1093-1096, 1979

Karmarkar NK, A new polynomial-time algorithm for linear programming, *Combinatorica*, 4, 373–395, 1984

The Mathematical Sputnik

组合优化 MATH T

- **The Mathematical Sputnik**
  - **The New York Times of November 7, 1979 announced an event which its readers could easily believe had the importance of the launching of Sputnik. "A surprise discovery by an obscure Soviet mathematician has rocked the world of mathematics and computer analysis ... Apart from its profound theoretical interest… the theory of codes could eventually be affected by the Russian discovery, and this fact has obvious importance to intelligence agencies everywhere".**
  - **In England, the Guardian broke the story three days earlier, under the headline, " Soviet Answer to 'Traveling Salesmen'. "**



Sputnik-1，苏联研制发射的世界第一颗人造地球卫星，1957年10月4日发射，开创了人类航天的新纪元

The New York Times

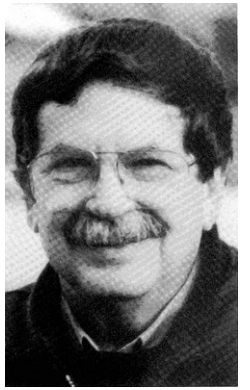
theguardian

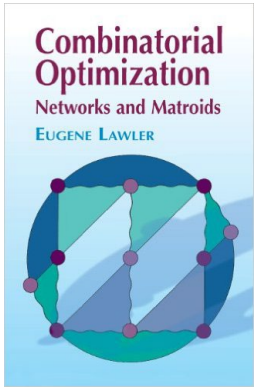
Lawler EL. The great mathematical Sputnik of 1979. *The Mathematical Intelligencer*, 2, 191-198, 1980.

The Mathematical Sputnik

组合优化
MATHT

- **The Mathematical Sputnik**
  - **Khachiyan emphatically did not discover a polynomial bounded algorithm for the TSP…What Khachiyan did do was to answer a much smaller question in complexity theory. Linear programming had been known to be a problem in $\mathcal{NP}$. It had not been shown to be $\mathcal{NP}$-complete…Khachiyan squeezed linear programming into $\mathcal{P}$ and thereby resolved the issue.**
  - **Only much later, on March 21, did the Times print a retraction,… But the headline read " A Russian's Solution in Math Questioned" and the subhead read " Americans Who Studied Khachiyan Linear Programming Method Express Doubt on Scope. " That made it sound as though poor Khachiyan had exaggerated the importance of his work, and Western mathematicians had cut him down to size.**





Eugene Leighton (Gene) Lawler
(1933-1994)
美国运筹学家

Lawler E. *Combinatorial Optimization: Networks and Matroids*, Dover Publications, 1976

整数线性规划



组合优化

MATHT

- 整数线性规划的求解
  - 划分问题 $\leq_m^p$ 0-1整数线性规划可行解存在性
    - 任取划分问题实例，正整数集 $A=\{a_1,a_2,\cdots,a_n\}$，构造仅含一个约束条件的0-1整数线性规划

$$\sum_{j=1}^{n} a_j x_j = \frac{1}{2}\sum_{j=1}^{n} a_j, x_j \in \{0,1\}$$

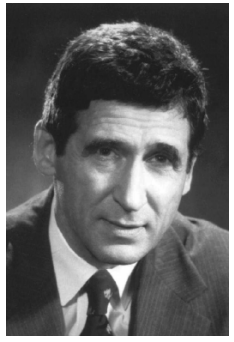
    - 整数线性规划有可行解当且仅当划分问题实例答案为“是”
  - 整数线性规划求解的判定形式是$\mathcal{NP}$-完全问题
    - 若线性不等式组 **Ax** ≥ **b** 有整数解，必存在一整数解，其规模不超过实例规模的多项式
  - 1960年左右，Gomory，Land和Doig分别给出了求解整数线性规划的割平面法（cutting plane method）与分支定界法（Branch and Bound）



Ailsa Horton Land (1927-2021)
英国数学家



Alison Grant Harcourt (neé Doig) (1929-)
澳大利亚数学家



Ralph Edward Gomory (1929-)
美国数学家

Papadimitriou CH. On the complexity of integer programming. *Journal of the ACM*, 28, 765-768, 1981

Land AH, Doig AG. An automatic method of solving discrete programming problems, *Econometrica* 28, 497–520, 1960.

ECONOMETRICA
JOURNAL OF THE ECONOMETRIC SOCIETY

整数线性规划



组合优化

MATH T

- 整数线性规划的松弛
  - 整数线性规划（IP）去除决策变量取整数约束后所得线性规划（LP）称为（IP）的松弛（relaxation）
    - （IP）的可行域包含于（LP）的可行域中
      - （IP）的可行解也是（LP）的可行解，但反之不然
      - （IP）的最优值不优于（LP）的最优值
    - （LP）的最优解与（IP）的最优解
      - 若（LP）的最优解为整数解，则它也是（IP）的最优解
      - 若（LP）的最优解不为整数解，不存在高效的取整策略将（LP）的最优解变为（IP）的最优解

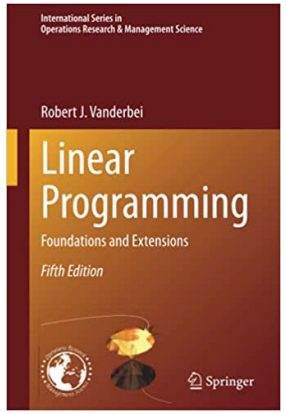
$$\text{(IP)}\quad \min\ \mathbf{cx}\quad s.t.\ \mathbf{Ax}=\mathbf{b},\ \mathbf{x}\in\mathbb{Z}_+^n$$

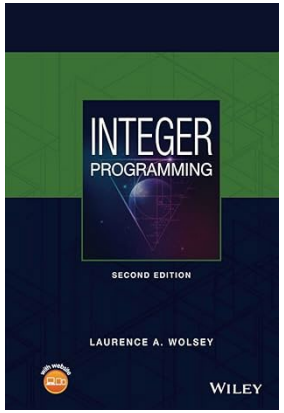
$$\text{(LP)}\quad \min\ \mathbf{cx}\quad s.t.\ \mathbf{Ax}=\mathbf{b},\ \mathbf{x}\in\mathbb{R}_+^n$$

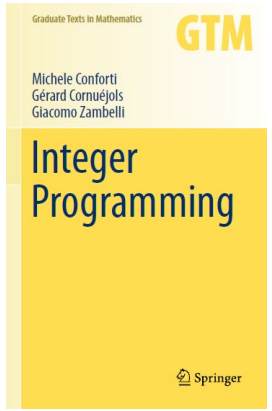
参考资料

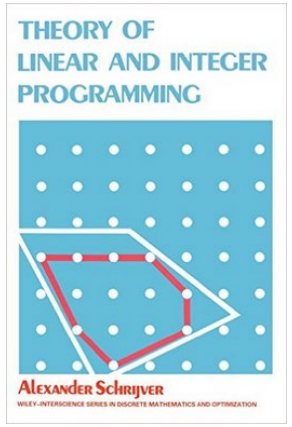
组合优化
MATH T

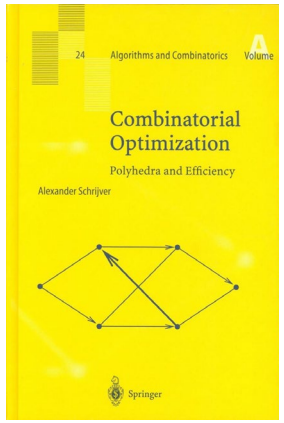


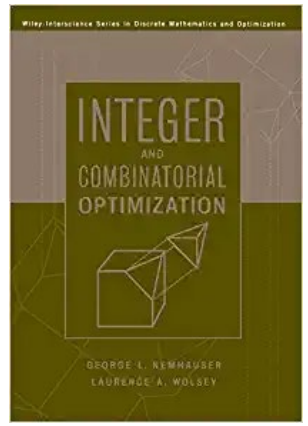












黄红选，韩继业，数学规划，清华大学出版社，2006

Vanderbei RJ, *Linear Programming(5th)*, Springer, 2020

Wolsey LA, *Integer Programming* (2nd), Wiley, 2020.

Conforti M, Cornuéjols G, Zambelli G, *Integer Programming*, Springer, 2014.

Schrijver A. *Theory of Linear and Integer Programming*. Wiley, 1998.

Schrijver A, *Combinatorial Optimization: Polyhedra and Efficiency*. Springer, 2003.

Wolsey LA, Nemhauser GL, *Integer and Combinatorial Optimization*, Wiley, 1999.


谢　谢